

S3C44B0 LINUX BOX PORTING GUIDE

고 성민 (nunorock@hanmail.net)

Chapter 1	개요	1
1.1	uClinux	1
1.2	준비물	1
1.2.1	HOST PC	2
1.2.2	TARGET BOARD	2
1.2.3	기타 장비	3
Chapter 2	개발 환경 설치	4
2.1	crosscompiler 설치	4
2.2	mlce 설치	6
2.2.1	Multi-Ice 설치 및 패치	6
2.2.2	Arm Debugger 설치 및 설정	7
2.2.3	mlce와 ADW 연결	8
2.2.4	Target 보드와 연결 RAM에서 실행	9
2.2.5	Target 보드 플래쉬에 write 하기	11
Chapter 3	부트로더	16
3.1	Blob	16
3.2	Blob 소스 수정 및 컴파일	17
3.2.1	Start	17
3.2.2	Rest	23
3.2.3	컴파일	34
3.2.4	Blob의 이용	37
Chapter 4	커널	39
4.1	소스	39
4.2	옵션설정과 컴파일	39
4.3	Target 보드에 저장	44
4.3.1	RAM에 커널이미지 다운로드	45
4.3.2	다운로드한 이미지를 Flash에 저장	45
4.3.3	커널 부팅	46

Chapter 5	루트파일 시스템	47
5.1	블록 디바이스 수정.....	47
5.2	롬디스크.....	48
Chapter 6	디바이스 드라이버	53
6.1	LRZSZ	53
6.1.1	소스 컴파일.....	53
6.1.2	테스트	54
6.2	MTD & JFFS	55
6.2.1	드라이버 작성.....	55
6.2.2	옵션메뉴스크립트 추가.....	61
6.2.3	테스트	67
6.2.4	부트시 자동 마운트	69
6.3	16C554 확장시리얼포트	70
6.3.1	드라이버 작성.....	71
6.3.2	커널옵션메뉴스크립트 추가.....	74
6.3.3	디바이스 노드 추가	76
6.3.4	테스트	76
6.4	FRAMEBUFFER.....	79
6.4.1	LCD spec	80
6.4.2	커널의 수정.....	80
6.4.3	프레임 버퍼 디바이스 노드 설정.....	82
6.4.4	테스트	82
6.5	KEYPAD.....	84
6.5.1	키패드 드라이버 작성	86
6.5.2	커널옵션메뉴스크립트 추가.....	89
6.5.3	디바이스 노드 추가	91
6.5.4	테스트	91
Chapter 7	PPPD 사용	94
7.1	수정 및 컴파일.....	94
7.2	실행.....	96

Chapter 1 개요

NSC4000L 단말기에서 PPP를 사용하기 위해 시도된 리눅스커널 포팅이었다. 결국 그 밖의 디바이스 드라이버까지 다 작성하게 되었지만 그리 순탄한 과정만은 아니었다. 수많은 시행착오를 거쳤지만 그 과정은 모두 생략하고 가장 간단하고 쉬운 방법의 과정만을 간추려 적기로 한다. 가장 쉽게 포팅할수 있는 방법으로 부트로더, 커널, 루트 파일 시스템, 디바이스 드라이버(MTD, 16C554, LCDFramBuffer, 키패드) 작성과 간단한 어플리케이션까지 작성해 보기로 하겠다. 그리고 조금 더 복잡한 방법은 작성에만도 상당한 시간이 걸리므로 차후 작성할 예정이다. (귀차니즘의 발동 ^ ^...)

1.1 uClinux

타겟보드의 삼성 S3C44B0 CPU는 MMU(Memory Management Unit)를 가지고 있지 않다. 그러므로 리눅스의 가상 메모리기능을 사용할수 없고 그로인해 메모리 보호 역시도 어려운 것이 사실이다. 일반적인 리눅스 커널을 그대로 사용할 수는 없다는 말이 된다.

그래서 이런 MMU가 없는 CPU에서도 가상 메모리 없이도 멀티 태스킹이 가능하도록 메모리부분을 대폭적으로 수정한 커널이 uClinux 커널이다.

uClinux의 u는 μ (micro)를 C는 (control)을 의미하며 이름에서도 알수 있듯이 MMU가 없는 MicroController 에서 사용할수 있도록 커널을 수정하였음을 알수 있다. 처음 Motorola CPU에 포팅이 된후 지금은 Arm, Intel, NEC등 다양한 Architecture에 포팅되어 사용되어 오고 있다.

이중 Arm 계열은 Samsung4530 까지 지원하나 S3C44B0는 지원하지 않고 있어 기존 uClinux-2.4.x 버전에서 S3C44B0를 추가 시켜주어야 한다. 그런데 중국에서 S3C44B0에 맞게 부트로더와 함께 커널패치를 해놓았다고 한다. 사용해본 결과 아주 만족스러웠다. 커널의 경우 거의 수정 없이 사용이 가능했다.

그래서 이 포팅문서 또한 2종류로 작성할 예정이다. 우선은 처음에 설명했듯이 이미 패치된 커널을 이용한 가장 빠른 방법으로 그 다음에는 일일이 패치를 다 해주는 방법을 택하기로 한다. (그걸 다 문서화 할 생각하면 ...쩍)

1.2 준비물

우선 가장 중요한 것은 uClinux를 포팅할 Target 보드가 있어야 할 것이고, 타겟보드와 통신하고 커널과 부트로더를 컴파일할 HOST PC가 한대 필요하다. 그리고 부트로더를 올릴 때 사용할 In-Circuit Emulator인 Multi-Ice가 필요하다. 그외에 ppp를 테스트 해보고 싶다면 TRS나 핸드폰이 필요하게 될것이다.

1.2.1 HOST PC

- OS

HANCOM LINUX 3.0 professional

Window2000 professional (Multi-Ice 사용시)

- TERMINAL

Minicom 2.1(linux) or HyperTerminal (M\$)

- TOOLS

arm-elf-tools-20030314 (linux arm cross compiler)

ARMSDT250 (M\$)

Multi-Ice 1.4 (M\$)

Dnw (M\$)

1.2.2 TARGET BOARD

- CPU

SAMSUNG S3C44B0 Arm7TDMI

- MEMORY

SAMSUNG 8 Mbyte SDRAM

Hynix 2 Mbyte X2 NOR Flash Memory

- 주변 장치

16C554 External Serial Port

320X200 Mono Graphic LCD

15 Key Keypad

GPS MODEM

- 메모리 맵

GCS7	0E00 0000h
GCS6 (DRAM 8MB) (0C00 0000 ~ 0C7F FFFFh)	0D00 0000h
GCS5 (16c554 Com4)	0A00 0000h
GCS4 (16c554 Com3)	0800 0000h
GCS3 (16c554 Com2)	0600 0000h
GCS2 (16c554 Com1)	0400 0000h
GCS1 (Flash1 2MB) (0200 0000 ~ 021F FFFFh)	0200 0000h
Special Registers	01C0 0000h
GCS0 (Flash0 2MB) (0000 0000 ~ 001F FFFFh)	0000 0000h

그림 1 Target Board의 메모리 맵

1.2.3 기타 장비

Multi-Ice

TRS

Car Kit 등등...

Chapter 2 개발 환경 설치

단말기용 프로그램 작성을 위해 가장 먼저 해야 할 일은 Cross Compiler의 설치이다. 그리고 더불어 minicom 같은 통신 에뮬레이터가 설치되어 있어야 하며 부트로더를 플래쉬메모리에 저장하기 위해 사용하는 Multi-Ice 같은 In-Circuit Emulator 사용 프로그램 또한 설치가 되어야 할 것이다.

2.1 crosscompiler 설치

Cross Compiler는 HOST PC에서 컴파일 하지만 그 결과 실행 파일은 ARM CPU 같이 다른 CPU상에서 실행가능토록 해주는 컴파일러이다.

GNU 의 gcc 컴파일러는 현존하는 다양한 종류의 CPU를 지원하고 있다. 물론 무료이고 그 안정성도 여러 개발자들에 의해 입증되어 오고 있으며 uClinux에서는 이 GNU의 크로스 컴파일러 툴을 사용하여 커널을 포팅하게 된다. 그중 ARM7TDMI용 Cross Compiler tool은 <http://www.uclinux.org/pub/uClinux/m68k-elf-tools/> 에서 구하여 쉽게 설치 할수 있다. 여기서는 2003년 10월 15일 현재 가장 최신 버전인 arm-elf-20030314를 사용하였다.

소스와 바이너리 두가지 버전을 모두 구할수 있으나 개요에서 언급한대로 이번에는 가장 쉽고 빠른 방법으로 설치할것이므로 일단 컴파일이 되어 있는 바이너리 버전을 다운 받아 설치한다. (모든 작업은 root 계정에서 하는것이 정신 건강에 도움이 될것이다.)

arm-elf-tools-20030314.sh 파일을 다운 받았으면 다음과 같이 설치한다.

➤ **sh arm-elf-tools-20030314.sh**

전에는 설치가 꽤 복잡했으나 그나마 그 수고도 덜어주고자 쉘 스크립트 파일로 설치를 용이하게 해놓았다. 설치가 완료되면 /usr/local/arm-elf 라는 파일이 디렉토리가 생성되어 있을 것이다.

Gcc를 비롯 각종 툴의 실행파일이 /usr/local/bin 에 설치가 되어 있을 것이다. 이 툴들을 컴파일시 사용할수 있도록 PATH에 툴의 실행디렉토리를 추가 지정해 준다.

➤ **vi /root/.bash_profile**

...

```
PATH=$PATH:$HOME/bin:/usr/local/bin
```

...

그리고 지정 PATH를 지금 적용하려면

➤ `source /root/.bash_profile`

과 같이 해주면 다시 로그인 해줄 필요 없이 PATH가 적용되어 당장 컴파일러를 사용할 수 있다.

설치된 컴파일러 파일들 중 많이 쓰이는 파일은 대략 표1과 같다.

파 일 이 름	설 명
arm-elf-gcc	GNU C compiler
arm-elf-c++	GNU C++ compiler
arm-elf-as	GNU assembler
arm-elf-gasp	GNU assembler preprocessor
arm-elf-ld	GNU linker
arm-elf-ar	Handles archives, librarian
arm-elf-objdump	Display object information
arm-elf-nm	List symbols from object files
arm-elf-strip	Discard symbols from object files
arm-elf-objcopy	Copy and translate object files
arm-elf-ranlib	Generate index to archives
arm-elf-size	List section size and total size
arm-elf-strings	Print the strings of printable characters in files
arm-elf-addr2line	Convert addresses into file names & linenums

표 1 /usr/local/bin 에 설치된 개발도구들

이상의 과정을 마치고 나면 어플리케이션을 컴파일 할 수 있게 된다. “Hello World” 와 같은 간단한 C 프로그램을 작성한 뒤에 arm-elf-gcc로 에러 없이 컴파일 되는지 확인해 보자.

소스코드 파일이름이 hello.c 라고 할 때 컴파일 명령은 다음과 같다.

```
# arm-elf-gcc -elf2flt -o hello hello.c
```

에러 메시지 없이 hello 라는 파일이 생성되었다면 컴파일러가 제대로 설치 되었다고 볼수 있다. 실행은 차후에 해보기로 하고 지금은 다음 단계로 넘어가기로 하자.

2.2 mlce 설치

Multi-Ice는 보드의 JTAG PORT를 통하여 프로세스를 직접 제어할수 있으므로 하드웨어적인 Breakpoint와 실시간 Trace 가 가능하다. 이 장비는 실제로는 소프트웨어를 디버깅하기 위한 목적으로 만들어진 장비가 아니라 하드웨어를 디버깅하기 위한 장비이다. 좀 불편하기는 하지만 소프트웨어를 디버깅하는데도 매우 강력하며 유용하다.

2.2.1 Multi-Ice 설치 및 패치

- ① Multi-Ice 1.4 버전을 설치한다.
- ② OS를 Windows98 이 아니라 Windows2000 을 사용할 경우에는 설치후 <http://www.arm.com/support/downloads-multi-ice> 에서 윈도우2000패치 파일 mice14w2kpatch.zip 을 다운 받아 압축을 풀어 multiice.sys 는 /Winnt/system32 에 복사해주고 multi-ICEServer.exe 는 M-ICE를 설치한 디렉토리에 덮어 쓰기 해준다.
- ③ M-ICE를 설치한 디렉토리에 있는 multi-ICE.dll 파일을 /Winnt/system32 (98의 경우 /Windows)에 복사를 해준다. 그리고 반드시 PC를 리부트 해준다.

2.2.2 Arm Debugger 설치 및 설정

① ARMSDT250 설치

- ② 메뉴에서 Option -> Debugger Configuration -> Target -> Add 를 선택
한후 M-ICE 설치시 복사해주었던 /Winnt/system32/Multi-ICE.dll 파일을
추가해준다. Multi-ICE 라는 타겟이 추가 되었을 것이다.

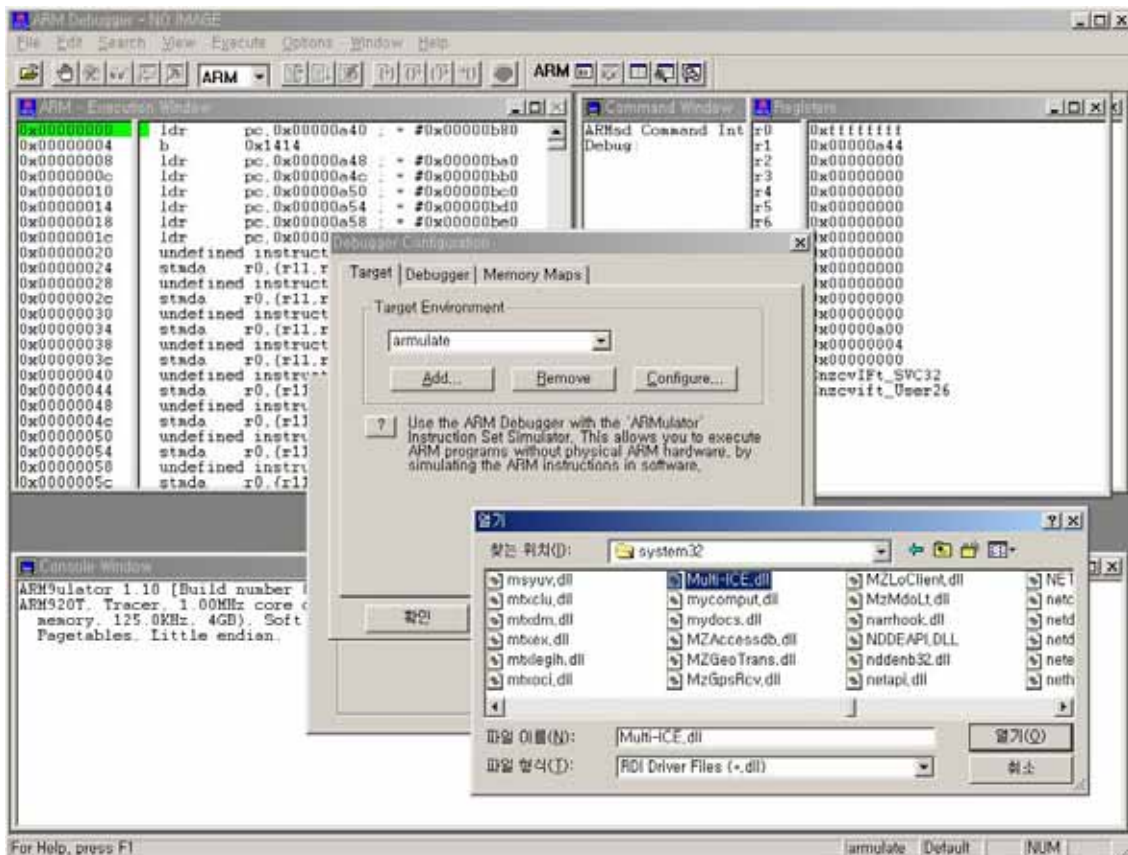


그림 2 ADW 설치 화면

2.2.3 mlce 와 ADW 연결

- ① Multilce와 타겟 보드를 연결하고 PC와 타겟 보드는 직렬포트로 연결, M-Ice와 PC는 병렬포트로 연결한다.
- ② 보드를 켜고 M-Ice 서버를 실행하고 서버에서 Ctrl-A를 누르거나 파일 메뉴에서 AutoConfigure를 해준다. [X] CPU Type(ARM7TDMI같은)이 뜨고 우측 하단 에 녹색불이 두개 들어 오면 정상이다.

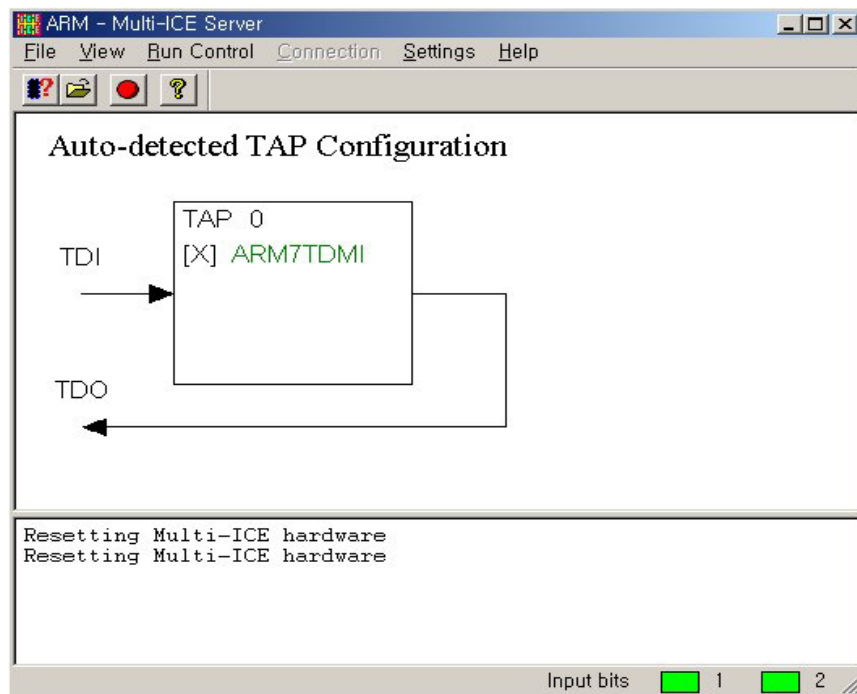


그림 3 Multi-Ice 설치 화면

- ③ ARM-Debugger를 실행하면 remote로 실행하겠냐는 메시지가 뜬다. 물론 예스를 선택하면 Configure debugger 의 configure 창이 뜨고 Processor Driver selection 에 M-ICE 에 연결된 CPU 타입(ARM7TDMI)이 나올것이다. OK 선택.

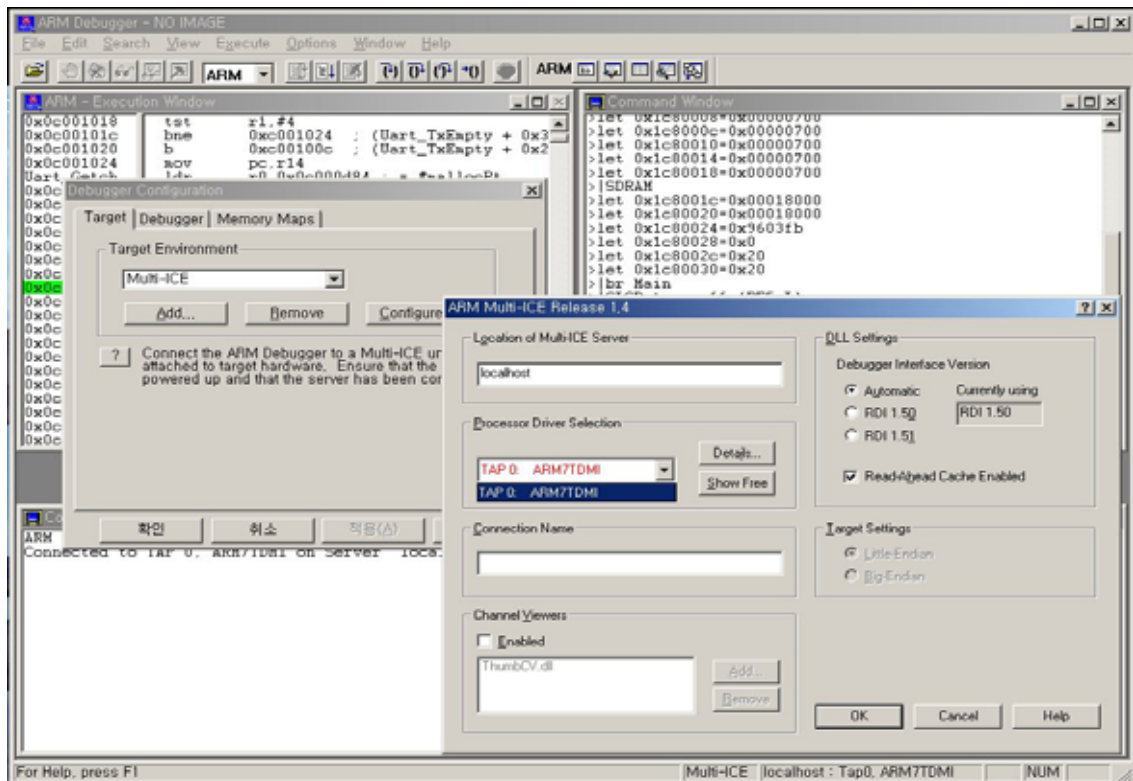


그림 4 mIce와 ADW 연결화면

2.2.4 Target 보드와 연결 RAM에서 실행

- ① 위에서와 같은 순서로 정상적으로 설치가 되었다면 이제 M-ICE를 이용하여 실행파일을 메모리에 복사하여 실행해보자
- ② 위과정이 모두 이상없이 이루어진 상태에서 삼성전자의 S3C44B0X 관련 사이트 (http://www.samsung.com/Products/Semiconductor/common/product.list.jsp?family_cd=LSI090101)에서 411s.ini 파일을 다운받아 사용하거나 아니면 이파일의 내용을 직접 command Window에 입력하여 보드를 초기화한다.

Debug : ob c:₩411s.ini

- ③ 중간에 에러가 없었다면 다음과 같이 복사할 파일 이미지를 로드한다.

Debug : load c:\W44btest.elf

- ④ 몇초간 파일을 복사할것이다. 데이터가 이동하고 있는지는 M-ICE의 Busy 램프가 깜빡이고 있는지 확인하면 된다. 모든 이미지가 복사되고 다시 프롬프트가 뜨면 실행시킨다.

Debug : go



```
Command Window
ARMsd Command Interface
Debug: ob c:\411s.ini
>let $vector_catch = 0x00
>let $semihosting_enabled = 0x00
>let psr=%IFt_SVC32
>|for ADW in ADS 1.0
>|let psr=%IFt_SVC
>|Fin=10MHz,Fout=60MHz,M/P/S=0x40/0x4/0x1
>let 0x1d80000=((0x40<<12)+(0x4<<4)+0x1)
>
>let 0x1c80000=0x11111110
>let 0x1c80004=0x00000700
>let 0x1c80008=0x00000700
>let 0x1c8000c=0x00000700
>let 0x1c80010=0x00000700
>let 0x1c80014=0x00000700
>let 0x1c80018=0x00000700
>|SDRAM
>let 0x1c8001c=0x00018000
>let 0x1c80020=0x00018000
>let 0x1c80024=0x9603fb
>let 0x1c80028=0x0
>let 0x1c8002c=0x20
>let 0x1c80030=0x20
>|br Main
>|CLCD turn_off (PE5=L)
>0x1d20028=0x400
>0x1d2002c=0x0
>
>
>
Debug: load c:\44btest.elf
Debug: go
```

그림 5 프로그램을 실행 명령어 입력

- 메모리에서 실행시키기 위해서는 플래쉬와 EEPROM 점퍼를 EEPROM으로 맞추어 주어야 한다 그렇지 않으면 플래쉬에 있는 이미지가 실행된다.

2.2.5 Target 보드 플래쉬에 write 하기

이제는 플래쉬 메모리에 이미지를 기록하는 방법을 보자.

- ① <http://www.samsung.com/Products/Semiconductor/SystemLSI/MobileSolutions/MobileASSP/MobileComputing/S3C44B0X/S3C44B0X.htm> 에서 DNW.exe 을 다운 받아 실행시키고 그림 6과 같이 Configuration -> option 에서 Baud rate을 115200으로 맞추고 PC와 연결된 시리얼 포트를 선택 후 Serial Port 메뉴에서 connect를 선택해 connect 한다.

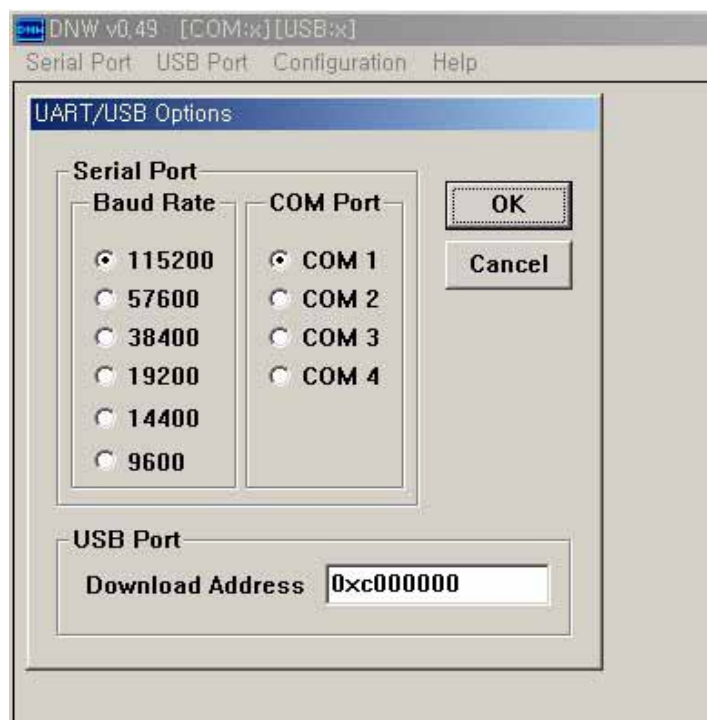


그림 6 DNW 설정화면

- ② 이제 위에서와 같은 방법으로 보드와 함께 제공된 44btest 프로그램을 실행시키자. (이실행프로그램은 플래쉬를 쓸수 있는 있어 이를 사용할것이다.) 그러면 DNW에 S3C44B0X의 테스트프로그램 메뉴가 보일것이다. 이 중 37번 메뉴가 Write Flash 메뉴이다. 37번을 선택하고 메모리선택 메뉴가 나오면 선택한다. (선택의 여지가 없이 하나이다)

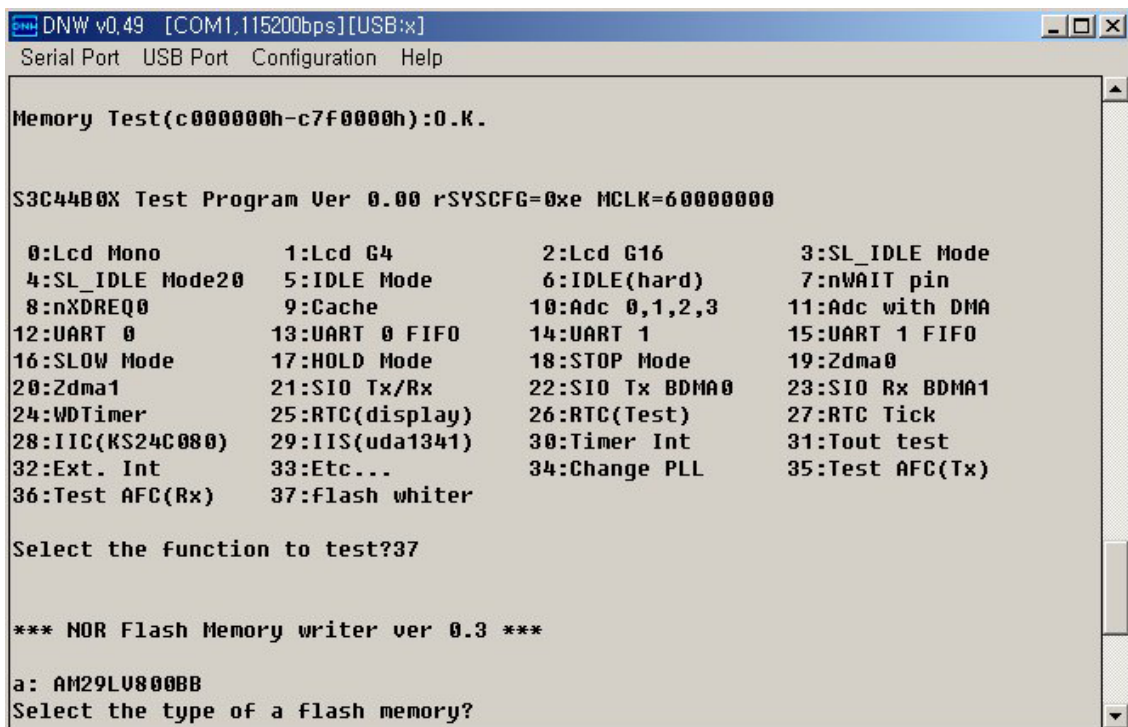
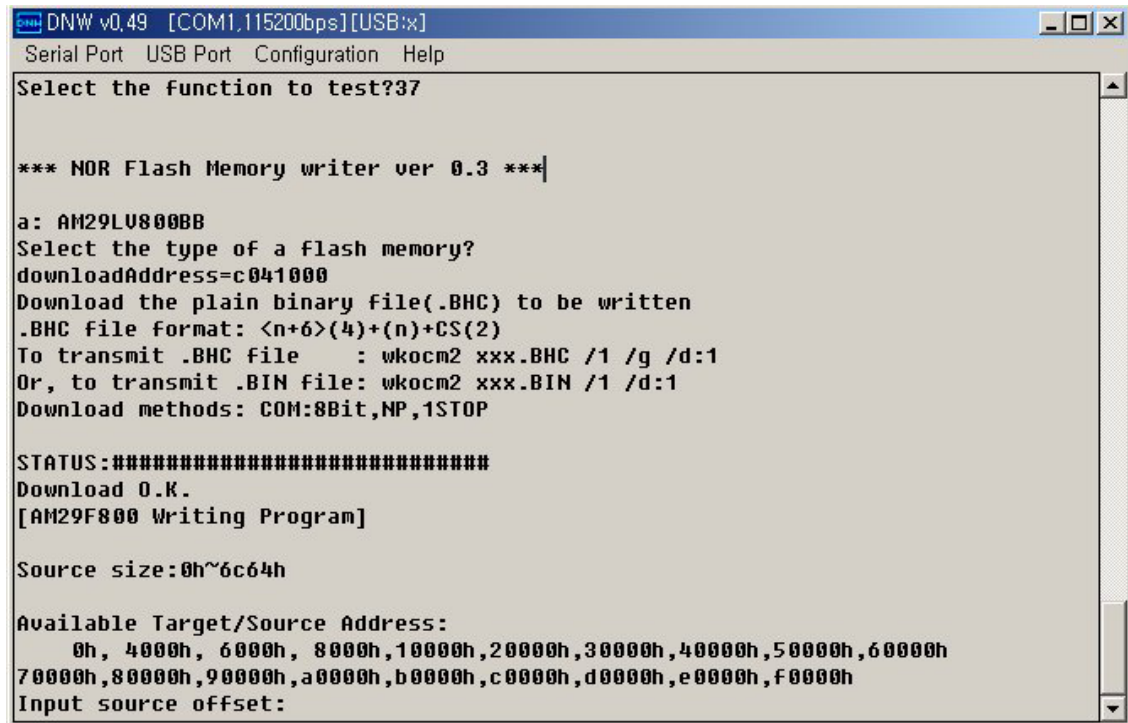


그림 7 44btest.elf 의 메인화면

- ③ 보드가 플래쉬에 write할 이미지 파일을 메모리에 올릴때까지 이 상태로 대기하게 된다. 이미지파일을 메모리에 올리기 위해서 DNW 좌측상단에 Serial Port메뉴에서 Transmitt을 선택하여 업로드할 이미지를 선택하면 다운로드 상태바가 나오면서 다운로드를 한다. (Wkcom2 파일을 이용할 수도 있는데 이 파일 역시 삼성 사이트에서도 구할수 있다. 그러나 윈도우95/98/Me 에서만 사용이 가능하고 DOS 프로그램이라 불편하니 쓰지 않는 것이... 굳이 쓴다면 다음과 같이 쓴다. C:\w> wkcom2 이미지파일 이름 /1 /d:1)

- ④ 다운 로드가 끝나면 다음과 같은 화면이 나온다.



```
DNW v0.49 [COM1,115200bps][USB:x]
Serial Port USB Port Configuration Help
Select the function to test?37

*** NOR Flash Memory writer ver 0.3 ***|

a: AM29LV800BB
Select the type of a flash memory?
downloadAddress=c041000
Download the plain binary file(.BHC) to be written
.BHC file format: <n+6>(4)+(n)+CS(2)
To transmit .BHC file      : wkocm2 xxx.BHC /1 /g /d:1
Or, to transmit .BIN file: wkocm2 xxx.BIN /1 /d:1
Download methods: COM:8Bit,NP,1STOP

STATUS:#####
Download O.K.
[AM29F800 Writing Program]

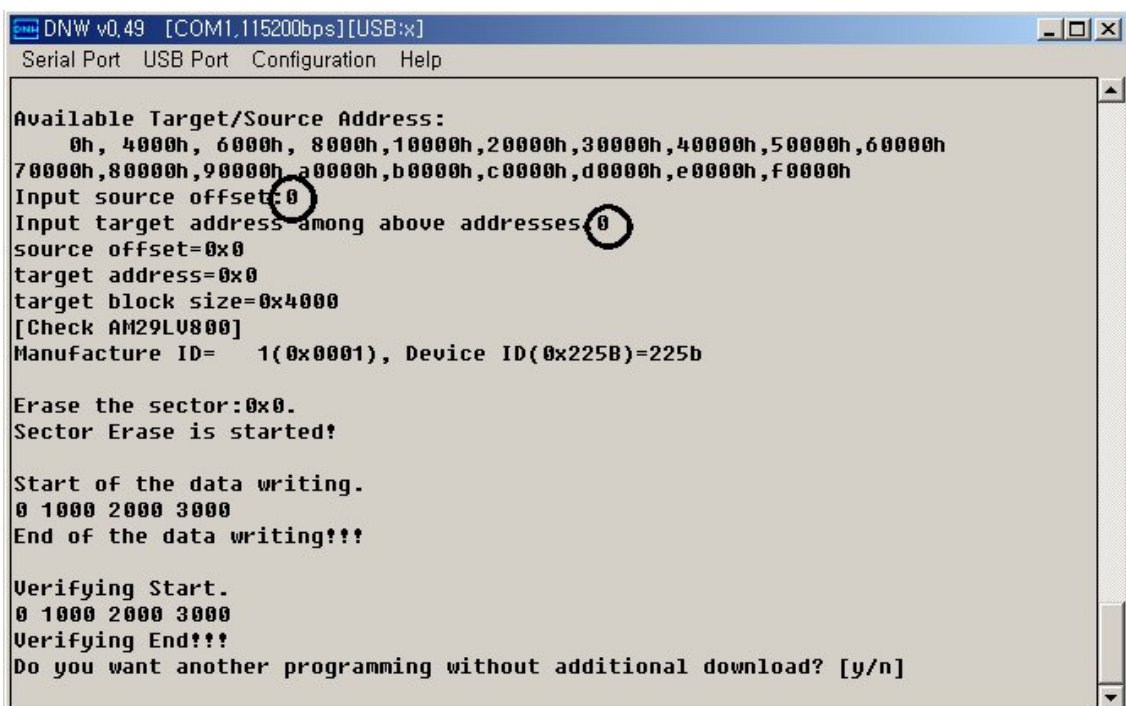
Source size:0h~6c64h

Available Target/Source Address:
    0h, 4000h, 6000h, 8000h,10000h,20000h,30000h,40000h,50000h,60000h
70000h,80000h,90000h,a0000h,b0000h,c0000h,d0000h,e0000h,f0000h
Input source offset:
```

그림 8 44btest.elf 의 flash write 화면

- ⑤ 메모리에 파일이 모두 전송되었으면 이제 플래쉬에 써넣을 차례이다. 소스 크기가 넘을때까지 source-offset 과 target-address 을 반복하여 입력한다. (그림8에서 처럼 소스의 크기가 6C64h일 경우 offset을 8000h 까지 입력해 주어야 한다.)

- ⑥ 처음에는 source offset은 '0h'부터 시작하고 target-address는 플래쉬 메모리에 write할 시작 주소값을 블록단위로 입력해주면 그 블록의 크기만큼 sector를 지우고 지운블록의 크기만큼의 다운로드된 이미지를 플래쉬에 write할것이다. (예제에서는 target-address가 0블록부터 시작했다. 플래쉬메모리는 erase를 블록 단위로만 할수 있어서 플래쉬의 첫번째 블록의 크기 4000h 만큼 플래쉬를 0~4000h 까지 지우고 다운로드이미지의 0~4000h 만큼 write한다.)



```
DNW v0.49 [COM1,115200bps][USB:x]
Serial Port USB Port Configuration Help

Available Target/Source Address:
    0h, 4000h, 6000h, 8000h, 10000h, 20000h, 30000h, 40000h, 50000h, 60000h,
    70000h, 80000h, 90000h, a0000h, b0000h, c0000h, d0000h, e0000h, f0000h
Input source offset: 0
Input target address among above addresses: 0
source offset=0x0
target address=0x0
target block size=0x4000
[Check AM29LV800]
Manufacture ID= 1(0x0001), Device ID(0x225B)=225b

Erase the sector:0x0.
Sector Erase is started?

Start of the data writing.
0 1000 2000 3000
End of the data writing!!!

Verifying Start.
0 1000 2000 3000
Verifying End!!!
Do you want another programming without additional download? [y/n]
```

그림 9 flash의 첫번째 블록에 write

- ⑦ 두번째 입력부터는 전 source-offset에 write한 target-address의 블록 크기를 더하여 입력하고 (0h+4000h=4000h) target-address에는 처음 입력한 주소에 write한 블록의 크기만큼 더하여 (0h+4000h) 입력한다.

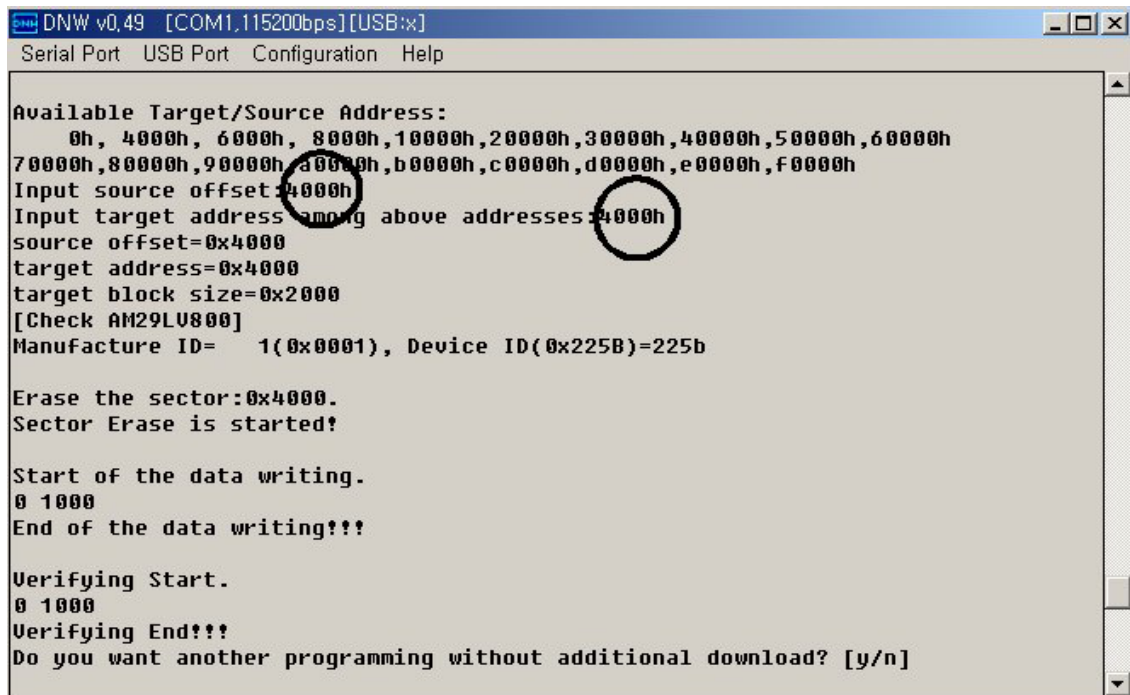


그림 10 flash 두번째 블록에 write

- ⑧ 이런식으로 source-offset이 파일 크기(6C64h)를 넘을때까지 write하면 flash에 write 완료. 그리고 flash로 부트하려면 dip스위치를 flash에 놓고 보드를 reset 한다.

Chapter 3 부트로더

단말기의 전원을 켜면 가장 먼저 실행되는 프로그램이 부트로더이다. 각 하드웨어 벤더들마다 각자의 부트로더를 제공하기도 하고 또 만들어 쓰기도 하지만 대부분 리눅스 커널을 사용할경우 Blob, redboot, bootldr 과 같은 오픈 소스 부트로더의 소스를 수정해서 많이들 사용한다. NSC4000L 리눅스 박스에서는 lart 의 오픈소스 프로그램인 BLOB을 수정하여 사용했다.

3.1 Blob

단말기의 전원이 켜지면 CPU는 물리주소 0번지에서 실행을 시작한다. 주소 0 번지에 플래쉬메모리를 맵핑시켜 놓으면 플래쉬 메모리 0 번지에서부터 프로그램 카운터가 시작되며, 롬을 맵핑시켜놓으면 롬 0 번지에서부터 프로그램이 시작된다. 그러므로 부트로더에서는 제일 먼저 메모리와 인터럽트등 기본적인 하드웨어 설정과 스택공간 할당등을 해주어야 한다. 그리고 커널이미지와 루트파일시스템 이미지를 플래쉬롬에서 램으로 복사하고 PC(Program Counter)를 복사한 커널영역으로 이동시켜주어 커널이 동작하기 위한 시작 코드의 역할도 해주게 된다. 부트로더의 가까운 예로서 데스크탑 PC에서는 부트로더역할을 롬바이오스와 GRUB (or LILO)가 있으며 임베디드 리눅스에서는 이에 더하여 플래쉬롬에 커널이미지와 롬디스크이미지를 기록할수 있는 기능이 더해지고 있다.

부트로더는 이제 시리얼 포트를 비롯한 중요한 하드웨어를 초기화한다. 시리얼 포트는 부트로더가 터미널 에뮬레이션 프로그램을 통하여 사용자와 대화하기 위한 용도로 사용된다. 하드웨어 초기화가 끝나면 시리얼 포트를 통해 부트로더 메시지가 출력되고 이때 사용자가 아무키도 누르지 않고 그대로 두면 램에 로드된 커널을 실행하게 된다. 그렇지 않고 어떤키를 눌러 BLOB 메뉴로 들어가면 부트로더와의 대화가 가능해지며 다음과 같은 작업을 할수 있다.

- Blob은 하드웨어(CPU 속도, 메모리, 인터럽트, RS232, I/O port)를 초기화한다.
- 리눅스 커널을 부팅하고 ramdisk를 제공한다.
- 커널이나 램디스크를 이미지를 타겟보드로 다운로드한다.

- 다운로드한 커널이나 램디스크이미지를 플래쉬메모리에 기록한다.
- 메모리맵을 설정하고 커널에 알려준다
- 커널에 command line을 준다.

이미지를 다운로드 시 저장되는 램의 영역과 플래쉬에 저장할때 사용되는 플래쉬의 주소 영역은 다음과 같다.

이미지	램 다운로드 영역	플래쉬 저장 영역	최대크기
BLOB	0x0C100000	0x00000000	64KB (0x010000)
커널	0x0C300000	0x00010000	512KB (0x080000)
롬디스크	0x0C400000	0x00090000	1472KB (0x170000)

표 2 각 이미지별 저장 위치

3.2 Blob 소스 수정 및 컴파일

Blob은 LART 홈페이지 <http://www.lart.tudelft.nl/lartware/blob/> 에서 구할수 있다. 여기서는 역시 가장 간단하고 빠른 방법으로 이 blob 소스를 S3C44B0X용으로 수정한 tpu의 BLOB 소스를 이용하기로 한다.

tpu의 BLOB 소스는 <http://www.start-web.net/tpu/download/blob-mba44b0.tgz> 에서 다운받을수 있다. 중국에서 S3C44B0X가 내장된 mba44b0보드용으로 blob 소스를 수정한것으로 직접 blob소스를 수정하는 것 보다 훨씬 일을 덜어줄것이다. 이 소스를 수정하여 NSC4000L용 부트로더를 만들기 위해 arm-elf 크로스 컴파일러로 컴파일을 해주기위해 약간의 수정작업이 필요하다. 우선 앞 과정의 개발환경의 설치를 선행한다. 그리고는 blob-mba44b0.tgz 다운 받아 압축을 풀고 다음과정을 따른다.

3.2.1 Start

Blob 소스코드는 크게 start와 rest로 나뉘어 이미지를 따로 생성한 다음 링크과정에서 하나로 합치고 있다. 이는 부트로더에서 실행코드를 메모리에 로드할 때

rest 부분을 따로 로드하기 때문에 코드 크기를 고정시키기 위한 방법이다. 따라서 소스의 수정도 편의상 두부분을 나눠서 하도록 하겠다.

- Start.S 수정

다음과 같이 I/O port 초기화 설정을 해준다.

타겟보드의 16C554, Flash, 키패드, GPS모뎀, LCD등의 주변 장치를 사용하기 위한 기본 I/O 설정들이다. 일부는 커널에 삽입는 것이 맞겠지만 제일 간단하고 빠른 방법을 사용하기로 했으므로 부트로더에 모두 포함시켰다.

각 I/O의 용도는 밑의 코드 1 의 주석을 참조 한다.

```
...
//-----
// NSC 4000L V1.0
// 2002-05-06
// OUTPORT: PULL UP DISADLE
// GND,VCC: INPUT PORT
// OUTPORT: rPUPX=1 , rPDATX=0
// INPORT: rPUPX=0 , rPDATX=1
//-----
/* Configuration Port Control Register*/
/* Port A */
//-----
// PORT A GROUP (0 = OUT, 1 = ADDRESS)
// ADDR24 ADDR23 ADDR22 ADDR21 ADDR20 ADDR19 ADDR18 ADDR17 ADDR16 ADDR0
// 0      0      1      1      1      1      1      1      1      1
//-----
    ldr    r1, =PCONA
    ldr    r0, =0x0ff
    str    r0, [r1]

    ldr    r1, =PDATA
    ldr    r0, =0x00
    str    r0, [r1]

/* Port B */
//-----
```

```

// PORT B GROUP (0 = OUT, 1 = IN)
// UARTE_CS UARTE_CS UARTE_CS UARTE_CS FLASH_CS POWER2 POWER1 nSRAS nSCAS SCLK SCKE
// 1      1      1      1      1      0      0      1      1      1      1
//-----

    ldr    r1, =PCONB
    ldr    r0, =0x7cf
    str    r0, [r1]

    ldr    r1, =PDATB
    ldr    r0, =0x030
    str    r0, [r1]

/* Port C */
//-----
// PORT C GROUP 00 = IN, 01 = OUT, 11 = Tx/Rx
// VCC VCC GPS_RXD1 GPS_RXD1 GND KO0 KO1 KO2 KO3 KO4 KO5 DISP/OFF GPS_RESET TCP_RESET
// TRSLED GPSLED
// 00  00 11      11      00 01 01 01 01 01 01 01      01      01
// 01  01
//-----

    ldr    r1, =PCONC
    ldr    r0, =0x0f155555
    str    r0, [r1]

    ldr    r1, =PDATC
    ldr    r0, =0x01f
    str    r0, [r1]
// LCD ON

    ldr    r1, =PUPC
    ldr    r0, =0xf7e0
    str    r0, [r1]

/* Port D */
//-----
// PORT D GROUP (00 = INPUT, 01 = OUTPUT, 10 = Vx, 11 = Reserved)
// VFRAME VM VLINE VCLK VD3 VD2 VD1 VD0

```

```

// 10      10 10      10      10      10      10      10
//-----

    ldr    r1, =PCOND
    ldr    r0, =0xaaaa
    str    r0, [r1]

    ldr    r1, =PUPD
    ldr    r0, =0xff
    str    r0, [r1]

    ldr    r1, =PDATD
    ldr    r0, =0x00
    str    r0, [r1]

/* Port E */
//-----
//PORT E GROUP 93C56 (00 = INPUT, 01 = OUTPUT, 10 = Rx/Tx, 11 = ??)
//GND  EEPDI  EEPDO  EEPCK  EEPCS  BUZZER  EXTO_RXD0  EXTO_TXD0  NC
//00   01    00    01     01    01      10          10          01
//-----

    ldr    r1, =PCONE
    ldr    r0, =0x04569
    str    r0, [r1]

    ldr    r1, =PDATE
    ldr    r0, =0x0
    str    r0, [r1]

    ldr    r1, =PUPE
    ldr    r0, =0x1b7
    str    r0, [r1]

/* Port F */
//-----
// PORT F GROUP Keyboard (00,000 = INPUT, 01,001 = OUTPUT)

```



```

// GND STLED KI2 KI1 NC NC NC KI0 VCC
// 000 001 000 000 01 01 01 00 00

//-----

    ldr    r1, =PCONF
    ldr    r0, =0x010150
    str    r0, [r1]

    ldr    r1, =PDATF
    ldr    r0, =0x00
    str    r0, [r1]

    ldr    r1, =PUPF
    ldr    r0, =0x100
    str    r0, [r1]

/* Port G */

//-----
// PORT G GROUP (00 = INPUT, 01 = OUTPUT, 10 = IIS, 11 = EINT)
// STLED2 STLED1 DOOR1 UART4_INT UART3_INT UART2_INT UART1_INT SPEED
// 01 01 11 11 11 11 11 11
//-----

    ldr    r1, =PCONG
    ldr    r0, =0x5fff // EINT7~0
    str    r0, [r1]

    ldr    r1, =PUPG
    ldr    r0, =0x0 // pull up enable
    str    r0, [r1]

/* SET ETC */
    ldr    r1, =SPUCR
    ldrb   r0, =0x7 // pull-up disable
    strb   r0, [r1]

    ldr    r1, =EXTINT
    ldr    r0, =0x22222222 // All EINT[7:0] will be falling edge triggered.

```

```

    str    r0, [r1]
...
    /* init LED */
//    bl    ledinit
...

```

코드 1 Configuration Port Control Register

- Memsetup-s3c44b0.S 수정

CPU속도 및 Endian 모드와 각 뱅크별 버스 폭과 크기 DRAM의 refresh 을 설정해준다.

Little endian 모드에 Dram과 FlashRom은 버스폭이 16비트로 연결되어 있으며 16c554는 8비트로 연결되어 있음을 알수 있다.

```

...
.text

MEMORY_CONFIG:
    .long 0x11000010
    .long 0x600
    .long 0x7ffc
    .long 0x7ffc
    .long 0x7ffc
    .long 0x7ffc
    .long 0x7ffc
    .long 0x18000
    .long 0x18000
    .long 0x9603fb
    .long 0x10
    .long 0x20
    .long 0x20

.globl memsetup
...

```

코드 2 Target 보드의 memory config

3.2.2 Rest

- Trampoline.S 수정

blob/src/trampoline.S 의 모든 cpsr-cfsx를 cpsr로 수정하고 bl main → bl Main 으로 수정해준다. cpsr-cfsx를 컴파일러가 인식을 못하므로 그냥 cpsr로 사용한다. ARM Developer 매뉴얼에 보면 cpsr로 사용해도 별 이상이 없는 것으로 보인다. 더불어 Blob은 커널상에서 실행 되는 프로그램이 아니어서 main()이라는 이름의 함수를 사용하면 에러 메시지가 출력 되고 컴파일이 안 되어 수정하였다.

- Mba44b0.h 수정 및 추가 (include/blob/arch/mba44b0.h)

NSC4000L의 플래쉬의 영역 구분과 각 special register들을 사용하기 위한 주소값등을 정의 해놓았다. Flash의 RAMDISK영역의 크기는 flash 크기에서 커널과 blob크기를 빼준 나머지 모두를 사용한다.

```
...  
  
#define KERNEL_FLASH_BASE (PARAM_FLASH_BASE + PARAM_FLASH_LEN)  
#define KERNEL_FLASH_LEN (512 * 1024)  
#define RAMDISK_FLASH_BASE (KERNEL_FLASH_BASE + KERNEL_FLASH_LEN)  
#define RAMDISK_FLASH_LEN (FLASH_SIZE - RAMDISK_FLASH_BASE)  
  
...
```

코드 3 Flash 메모리영역 수정

다음은 파일 끝에 추가해준다.

__cli()는 모든 인터럽트를 중단시키는 인라인 어셈블이고 __sti()가시 인터럽트를 활성화 시켜준다. __save_flags(x) 는 __cli() 실행시 x변수에 status register값을 임시 저장하고 __sti() 실행시에 임시 저장값을 다시 로드해준다. 이는 타이머 인터럽트로 인해 플래쉬가 제대로 erase가 안되는 경향이 있는 것 같아 커널소스를 참조해왔다.

```
...  
  
/* FLASH0 size */  
#define FLASH_SIZE (0x00200000)  
  
#define FTIMEOUT 990000000  
  
/*
```

```

* Enable IRQs
*/
#define __sti()                                W
    ({                                        W
        unsigned long temp;                W
        __asm__ __volatile__(              W
            "mrs    %0, cpsr                @ sti\n"    W
"        bic      %0, %0, #128\n"          W
"        msr      cpsr_c, %0"              W
: "=r" (temp)                               W
:                                           W
: "memory");                               W
    })

/*
* Disable IRQs
*/
#define __cli()                                W
    ({                                        W
        unsigned long temp;                W
        __asm__ __volatile__(              W
            "mrs    %0, cpsr                @ cli\n"    W
"        orr      %0, %0, #128\n"          W
"        msr      cpsr_c, %0"              W
: "=r" (temp)                               W
:                                           W
: "memory");                               W
    })

/*
* save current IRQ & FIQ state
*/
#define __save_flags(x)                        W
    __asm__ __volatile__(                    W
        "mrs      %0, cpsr                  @ save_flags\n"    W
: "=r" (x)                                   W

```

```

:                                     W
: "memory")

/*
 * restore saved IRQ & FIQ state
 */
#define __restore_flags(x)           W
    __asm__ __volatile__(           W
    "msr    cpsr_c, %0              @ restore_flags\n"   W
    :                                     W
    : "r" (x)                        W
    : "memory")
...

```

코드 4 NSC4000L의 system 헤더파일

- Main.c 수정

Trampoline.S 에서 bss clear와 cpu의 각모드에서의 스택 포인터 지정후 Main()으로 점프하도록 수정하였으므로 main.c 의 main()함수를 Main()으로 수정해준다.

Do_reload는 flash에 저장되어 있는 blob과 kernel, ramdisk 이미지를 RAM으로 복사하여 실행토록 해준다.

```

...
static int do_reload(char *what)
{
...
/*
    if(ptype == 0){
        MyMemCpy(dst, src, numWords);
    }else{
        nand_read(dst, src, numWords);
    }
*/
    MyMemCpy(dst, src, numWords);

```

```

        SerialOutputString(" done\n");

        return 0;
    }
    ...

```

코드 5 do_reload의 수정

원 소스에서 사용하고 있는 플래쉬는 nand 플래쉬이므로 Target 보드에서 쓰는 NOR 플래쉬를 사용하기 위해 다음과 같이 수정해준다. 플래쉬 영역을 Noncachable로 지정해주지 않을 경우 플래쉬에 command를 보내면 cache가 먼저 동작하여 erase가 제대로 실행이 되지 않는다. 이는 후에 16c554를 사용할 경우도 마찬가지로 반드시 Noncachable 영역으로 지정해주어야한다.

```

...
static int Flash(int argc, char *argv[])
{
    u32 numBytes = 0;
    u32 maxSize = 0;
    u32 nwords;
    block_source_t type = fromFlash;
    int ptype,i;

    /* flash area(Bank0) must be non-cachable area. */
    REG(NCACHBE0) =( (0x2000000>>12)<<16 )|(0>>12);

    if(argc < 2)
        return -ENOPARAMS;

    if(strncmp(argv[1], "blob", 5) == 0) {

        SerialOutputString("\nWARNING!!! Don't touch the flash memory while
writing bootloader! \n");
        SerialOutputString("Erase the Blob Sector...");
        REG(TCON) = 0x04000000;
    }
}

```

```

flash_amd16_erase(0x0);
flash_amd16_erase(0x4000);
flash_amd16_erase(0x6000);
flash_amd16_erase(0x8000);

    if(!BlankCheck(0x0,0x10000))
    {
        SerialOutputString("    [FAIL!] \n");
        return -EFLASHERASE;
    } else SerialOutputString("    [SUCCESS] \n");

    for(i=0;i<BLOB_FLASH_LEN;i+=2)
    {
        flash_amd16_write(    i+BLOB_FLASH_BASE,    *(    (u16    *)
(BLOB_RAM_BASE+i) ) );
    }

    REG(TCON) = 0x05000000;

    maxSize = BLOB_FLASH_LEN;
    numBytes = blob_status.blobSize;
    type = blob_status.blobType;
    ptype = 0;

#ifdef PARAM_START
    } else if(strncmp(argv[1], "param", 6) == 0) {
        src = (u32 *)PARAM_RAM_BASE;
        dst = (u32 *)PARAM_FLASH_BASE;
        maxSize = PARAM_FLASH_LEN;
        numBytes = blob_status.paramSize;
        type = blob_status.paramType;
        ptype = 0;
#endif

    } else if(strncmp(argv[1], "kernel", 7) == 0) {
        for (i=KERNEL_FLASH_BASE ; i<RAMDISK_FLASH_BASE ; i+=0x10000)

```

```

{
    SerialOutputString("  Start erase block ");
    SerialOutputHex(i);
    flash_amd16_erase(i);
    if(!BlankCheck(i,0x10000))
    {
        SerialOutputString("    [FAIL!] \n");
        return -EFLASHERASE;
    } else SerialOutputString("    [SUCCESS] \n");
}

    SerialOutputString("\nWrite the Kernel Block... \n");

    for(i=0;i<KERNEL_FLASH_LEN;i+=2)
    {
        flash_amd16_write( i+KERNEL_FLASH_BASE,* (u16
*)(KERNEL_RAM_BASE+i) );
        if((i%0x10000)==0)
        {
            SerialOutputHex(i);
            SerialOutputString(" ");
        }

    }

    numBytes = blob_status.kernelSize;
    maxSize = KERNEL_FLASH_LEN;
    type = blob_status.kernelType;
    ptype = 0;

} else if(strncmp(argv[1], "ramdisk", 8) == 0) {

    SerialOutputString("\nErase the RAMDISK Block... \n");

    for (i=RAMDISK_FLASH_BASE ; i<FLASH_SIZE ; i+=0x10000)
    {

```



```

        SerialOutputString("  Start erase block ");
        SerialOutputHex(i);

        flash_amd16_erase(i);

        if(!BlankCheck(i,0x10000))
        {
            SerialOutputString("    [FAIL!] \n");
            return -EFLASHERASE;
        } else SerialOutputString("    [SUCCESS] \n");
    }

    SerialOutputString("\nWrite the RAMDISK Block... \n");

    for(i=0;i<RAMDISK_FLASH_LEN;i+=2)
    {
        flash_amd16_write(            i+RAMDISK_FLASH_BASE,*          (u16
*)(RAMDISK_RAM_BASE+i) ) );
        if((i%0x10000)==0)
        {
            SerialOutputHex(i);
            SerialOutputString(" ");
        }

    }

    numBytes = blob_status.ramdiskSize;
    maxSize = RAMDISK_FLASH_LEN;
    type = blob_status.ramdiskType;
    ptype = 0;
} else {
    printerror(EINVAL, argv[1]);
    return 0;
}

if(type == fromFlash) {
    printerrprefix();

```

```

        SerialOutputString(argv[1]);
        SerialOutputString(" not downloaded\n");
        return -EINVAL;
    }

    if(numBytes > maxSize) {
        printerrprefix();
        SerialOutputString("image too large for flash: 0x");
        SerialOutputHex(numBytes);
        SerialOutputString(" > 0x");
        SerialOutputHex(maxSize);
        serial_write('W\n');

        return -ETOOLONG;
    }

    nwords = (numBytes + sizeof(u32) - 1) / sizeof(u32);

    SerialOutputString("Saving ");
    SerialOutputString(argv[1]);
    SerialOutputString(" to flash\n");

    /*
        if(ptype==0){
            atmel_write(dst, src, nwords);
        }else{
            nand_write(dst, src, nwords);
            flash_write_region(dst, src, nwords);
        }
    */
    REG(NCACHBE0) = 0x0;

    return 0;
}
...

```

코드 6 Flash() 함수의 수정

Flash 메모리를 지우고 쓰는 함수들이다.

```
...
/*
 *   FLASH blank check routine
 */
int BlankCheck(int targetAddr,int targetSize)
{
    int i,j;
    for(i=0;i<targetSize;i+=2)
    {
        j=((unsigned short *)(i+targetAddr));
        if( j!=0xffff)
        {
            SerialOutputString("E:");
            SerialOutputHex(i+targetAddr);
            SerialOutputString("=");
            SerialOutputHex(j);
            SerialOutputString(" \n");
            return 0;
        }
    }
    return 1;
}

/*
 * FLASH identify routine for the AMD 29LVX00 family devices.
 */
void flash_amd16_test(void)
{
    unsigned long fbase = 0x00000000;
    unsigned short manID, devID;

    *((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xaaaa;
    *((volatile unsigned short *) (fbase | (0x2aa << 1))) = 0x5555;
```

```

*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0x9090;

manID = (*((volatile unsigned short *) (fbase << 1)));

*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xaaaa;
*((volatile unsigned short *) (fbase | (0x2aa << 1))) = 0x5555;
*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0x9090;

devID = (*((volatile unsigned short *) (fbase | (0x001 << 1))));

*((unsigned volatile short *) fbase) = 0xf0f0; /* Reset */

SerialOutputString("Manufacture ID=");
SerialOutputHex(manID);
SerialOutputString(" Device ID=");
SerialOutputHex(devID);
SerialOutputString("\n");

}

/*
 *   FLASH erase routine for the AMD 29LVX00 family devices.
 */
static void flash_amd16_erase(unsigned long t_addr)
{
    unsigned volatile short *address;
    unsigned long fbase = 0x00000000;
    unsigned long flags;
    unsigned short status;
    int i;

    address = (unsigned volatile short *) (fbase + t_addr);

    __save_flags(flags); __cli();

```

```

/* Erase this sector */
*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xaaaa;
*((volatile unsigned short *) (fbase | (0x2aa << 1))) = 0x5555;
*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0x8080;
*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xaaaa;
*((volatile unsigned short *) (fbase | (0x2aa << 1))) = 0x5555;
*address = 0x3030;

for (i = 0; (i < FTIMEOUT); i++) {
    status = *address;
    if ((status & 0x0080) || (status & 0x0020))
        break;
}

if (*address != 0xffff) {
    SerialOutputString("Flash Erase Failed!!!");
    *((unsigned volatile short *) fbase) = 0xf0f0; /* Reset */
}

__restore_flags(flags);

}

/*
 *   FLASH programming routine for the 29LVX00 device family.
 */
static void flash_amd16_write(unsigned long t_addr, unsigned short buffer)
{
    unsigned volatile short *address;
    unsigned long fbase = 0x00000000;
    unsigned short status;
    unsigned long flags;
    int i;

    address = (unsigned volatile short *) (fbase + t_addr);
    __save_flags(flags); __cli();

```

```

*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xaaaa;
*((volatile unsigned short *) (fbase | (0x2aa << 1))) = 0x5555;
*((volatile unsigned short *) (fbase | (0x555 << 1))) = 0xa0a0;
*address = buffer;

for (i = 0; (i < FTIMEOUT); i++) {
    status = *address;
    if (status == buffer) {
        /* Program complete */
        break;
    }
}

if (*address != buffer) {
    SerialOutputString("Flash Write Failed!!!");
    *((unsigned volatile short *) fbase) = 0xf0f0; /* Reset */
}

__restore_flags(flags);

}
...

```

코드 7 FlashRom 조작 함수

3.2.3 컴파일

- Configure.in 수정

Configure에서 컴파일 옵션을 설정해주므로 약간의 수정이 필요하다.

```

...
AC_CHECK_PROGS(OBJCOPY, arm-linux-objcopy objcopy, echo)
AC_CHECK_PROGS(RANLIB, arm-linux-ranlib ranlib, echo)
AC_CHECK_PROGS(AR, arm-linux-ar ar, echo)
AC_CHECK_PROGS(NM, arm-linux-nm nm, echo)

```

```

...
1CFLAGS='echo $CFLAGS | sed 's/-O2//''

CFLAGS='echo $CFLAGS -march=armv4 -mtune=strongarm1100 -fomit-frame-
pointer -fno-builtin -mapcs-32 -nostdinc`
LDFLAGS='echo -static -nostdlib`
OCFLAGS="-O binary -R .note -R .comment -S"trap " 1 2 15
cat > confcache <<WEOF
...

```

코드 8 configure.in

LDLAGS는 configure 시 컴파일러 검사를 하게 되는데 간단한 소스를 컴파일 하여 테스트를 해본다. 이때 flat 메모리를 사용하는 uclinux에서 -elf2flt 옵션을 지정해주지 않으면 컴파일러 테스트에 실패 하므로 일단 테스트에 통과 할수 있도록 -elf2flt 옵션을 넣어주되 실제 부트로더 컴파일시에는 적용이 되지 않도록 하기 위해 수정해준다. (부트로더는 커널위에서 실행되는 파일이 아니므로 -elf2flt 옵션을 적용해주어서는 안된다.)

- Makefile.am 설정

blob은 초기화 부분인 start와 나머지 부분인 rest가 합쳐져서 최종 실행 파일이 만들어진다. 소스분석을 하면서 메모리 맵이 어떻게 되는지 궁금하면 Makefile.in을 약간 수정하여 플래쉬와 램에 각 함수들이 어떻게 배치되어 있는지 볼수 있다.

¹ 컴파일시 옵티마이즈 옵션을 적용 주기 위한 수정으로 특정 컴파일러에서는 옵티마이즈를 할 경우 실행 파일의 사이즈는 줄어드나 플래쉬 명령(CFI)를 제대로 인식하지 못하는 경우도 있어 수정했으나 수정하지 않아도 cache 설정만 잘해준다면 문제가 없을 것으로 보인다.

```

...
blob-start-elf32: $(blob_start_elf32_OBJECTS) $(blob_start_elf32_DEPENDENCIES)
    @rm -f blob-start-elf32
    $(LINK)      $(blob_start_elf32_LDFLAGS)      $(blob_start_elf32_OBJECTS)
$(blob_start_elf32_LDADD) $(LIBS)
    $(NM) - -numeric-sort $@ > start.map

blob-start-chain-elf32:                                $(blob_start_chain_elf32_OBJECTS)
$(blob_start_chain_elf32_DEPENDENCIES)
    @rm -f blob-start-chain-elf32
    $(LINK) $(blob_start_chain_elf32_LDFLAGS) $(blob_start_chain_elf32_OBJECTS)
$(blob_start_chain_elf32_LDADD) $(LIBS)

blob-rest-elf32: $(blob_rest_elf32_OBJECTS) $(blob_rest_elf32_DEPENDENCIES)
    @rm -f blob-rest-elf32
    $(LINK)      $(blob_rest_elf32_LDFLAGS)      $(blob_rest_elf32_OBJECTS)
$(blob_rest_elf32_LDADD) $(LIBS)
    $(NM) - -numeric-sort $@ > rest.map
...

```

코드 9 Makefile.am 의 수정

- Configuration 설정

```
# 2automake
```

```
# autoconf
```

```
# 3 CC=arm-elf-gcc OBJCOPY=arm-elf-objcopy RANLIB=arm-elf-ranlib
AR=arm-elf-ar NM=arm-elf-nm LDFLAGS=-elf2flt ./configure 4 - -with-
linux-prefix=/root/Nsc4kl/kernel/uClinux-2.4.x - -with-board=mba-44b0
```

- 컴파일

```
# make
```

이상으로 어려없이 컴파일이 잘 되었다면 ~src/blob 디렉토리에 blob 이라는 바이너리 이미지가 생성된다. 이를 2.2에서 설명한대로 Multi-Ice의 플래쉬 0번지에

² Automake는 반드시 automake-1.4-p5를 다운받아 컴파일 하여 사용해야 한다.

³ 다시 configuration 을 해결경우는 config.cache 파일을 지우고 해줘야 기존 config값이 로드되지 않는다.

⁴ 커널소스의 디렉토리를 지정해준다.

write한다.

3.2.4 Blob 의 이용

터미널 프로그램(win:hyper terminal, linux:minicom)을 115200 baud, 8 data bits, no parity, 1stop bit, no start bits로 세팅하여 타겟보드의 시리얼과 연결한다. Target 보드의 전원을 넣으면키면 그림 11과 같은 화면이 출력되며 부트로더와 커널, 롬디스크 이미지를 플래쉬 메모리에서 램으로 LOAD 해준다..

Consider yourself LARTed!

blob version 2.0.5-pre2 for Samsung MBA-44B0 Board

Copyright (C) 1999 2000 2001 Jan-Derk Bakker and Erik Mouw

blob comes with ABSOLUTELY NO WARRANTY; read the GNU GPL for details.

This is free software, and you are welcome to redistribute it

under certain conditions; read the GNU GPL for details.

Loading blob from flash . done

Loading kernel from flash .. done

Loading ramdisk from flash done

Autoboot in progress, press any key to stop ...

...

그림 11 BLOB의 메인화면

이때 만일 5초 동안 아무키도 누르지 않는다면 자동으로 커널을 실행하게 되지만 아무키나 누르면 다음과 같은 프롬프트가 출력된다.

...

Autoboot in progress, press any key to stop ..

Autoboot aborted

Type "help" to get a list of commands

blob>

그림 12 BLOB Prompt

이 상태에서 수행할수 있는 기본적인 기능은 다음과 같다.

```
...
blob> help
The following commands are supported:
* reset
* boot  #리눅스 커널을 부팅
* dm    #메모리 덤프
* ne     #nand flash erase
* nr     #nand flash read
* nw     #nand flash write
* xdownload  # 이미지 파일 다운로드
* flash  # 다운로드 받은 이미지 파일 플래쉬 메모리에 기록
* speed  # 시리얼 baud speed 지정
* status  # blob 상태 표시
* reload  # 이미지 리로드
* reblob  # blob 리로드
* help
Use "help command" to get help on a specific command
blob> help flash
Help for 'flash':

Usage: flash {blob|param|kernel|ramdisk}
Write <argument> image to flash
blob> help xdownload
Help for 'xdownload':

Usage: xdownload {blob|param|kernel|ramdisk}
Download <argument> image to RAM using xmodem
blob>
```

그림 13 BLOB 명령어 도움말

* help 명령은 부트로더가 지원하는 모든 명령어와 각각에 대한 간단한 설명과 문법을 출력해준다.

Chapter 4 커널

부트로더까지 잘 올라갔다면 이제는 본 게임인 커널을 올려 줄 차례이다. 커널 소스를 컴파일하여 이미지가 생성되면 BLOB을 이용하여 단말기에 로드하고 플래쉬 메모리에 저장할수 있게 된다.

4.1 소스

www.uclinux.org 에서 uclinux 커널을 다운 받아 소스를 수정해주어야 한다. 그러나 역시 많은 시간과 노력을 요구하므로 tpu의 소스를 다운 받아 사용하기로 한다. Tpu의 커널소스는 전혀 수정할 필요가 없다. 단지 옵션만 잘 선택해주고 컴파일만 잘해주면 된다.

패치된 커널소스는 Blob 소스를 구했던 <http://www.start-web.net/tpu/> 에서 구할 수있다.

4.2 옵션설정과 컴파일

패치된 커널 소스 (uClinux-2.4.17-uc1.tgz)를 다운 받았으면 적당한 디렉토리에 커널압축을 푼다.

➤ `tar zxvf uClinux-2.4.17-uc1.tgz`

압축을 풀면 uClinux-2.4.x 라는 디렉토리가 생성 되어 있을 것이고 이 디렉토리에 패치된 커널 소스가 있을것이다. 그럼 이 디렉토리로 들어가 옵션을 설정해주어야 한다.

그전에 RAM과 FLASH의 주소와 크기를 NSC4000L 단말기에 맞게 수정한다.

~/arch/armnommu/config.in 의 일부를 다음과 같이 RAM 8MB와 FLASH 2MB로 수정한다.

```
...
if [ "$CONFIG_ARCH_S3C44B0" = "y" ]; then
    define_bool CONFIG_NO_PGT_CACHE y
    define_bool CONFIG_CPU_32 y
```

```

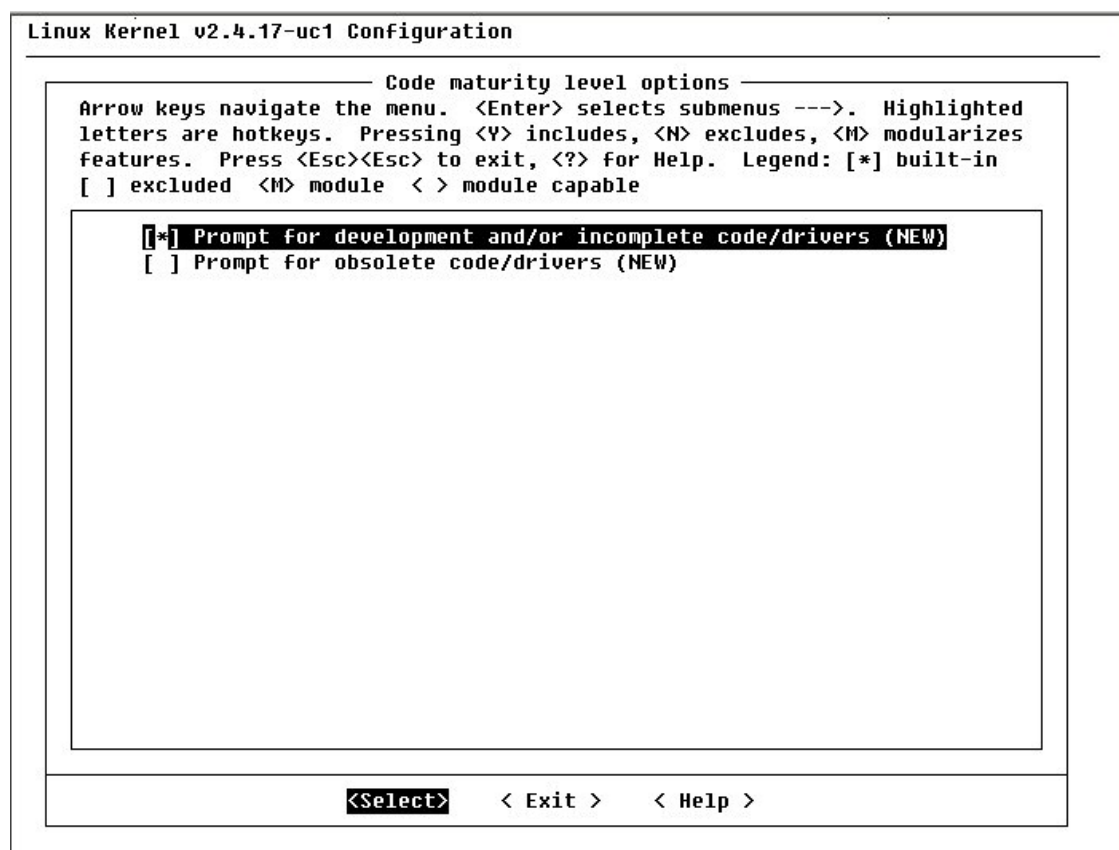
define_bool CONFIG_CPU_26 n
define_bool CONFIG_CPU_ARM710 y
define_bool CONFIG_CPU_WITH_CACHE y
define_bool CONFIG_CPU_WITH_MCR_INSTRUCTION n
define_hex DRAM_BASE 0x0c000000
define_hex DRAM_SIZE 0x00800000
define_hex FLASH_MEM_BASE 0x00000000
define_hex FLASH_SIZE 0x00200000
fi
...

```

코드 10 Dram과 FlashMemory 주소값 및 크기 지정

➤ make menuconfig

다음과 같이 옵션을 설정한다.



Linux Kernel v2.4.17-uc1 Configuration

General setup

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in [] excluded <M> module < > module capable

```
[ ] Support hot-pluggable devices
[*] Networking support
[*] System V IPC
[ ] Reduced memory footprint
[ ] BSD Process Accounting
[*] Sysctl support
<*> MMIO math emulation
(ELF) Kernel core (/proc/kcore) format
[ ] Power Management support
< > RISC OS personality
[ ] Kernel-mode alignment trap handler
```

<Select> < Exit > < Help >

Linux Kernel v2.4.17-uc1 Configuration

System Type

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in [] excluded <M> module < > module capable

```
(S0C44B0) ARM system type
[ ] Generate big endian code
[*] Set flash/sdram size and base addr
(0c000000) (S)DRAM Base Address
(00800000) (S)DRAM Size
(00000000) FLASH Base Address
(00200000) FLASH Size
```

<Select> < Exit > < Help >

Linux Kernel v2.4.17-uc1 Configuration

Networking options

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in [] excluded <M> module < > module capable

```
[ ] Packet socket
[ ] Netlink device emulation
[ ] Network packet filtering (replaces ipchains)
[*] Socket Filtering (NEW) (EXPERIMENTAL) (NEW)
[ ] Unix domain sockets (NEW) (EXPERIMENTAL) (NEW)
[*] TCP/IP networking over IP (NEW)
[ ]   IP: multicasting (NEW) (EXPERIMENTAL) (NEW)
[ ]   IP: advanced router (NEW) (EXPERIMENTAL) (NEW)
[ ]   IP: kernel level autoconfiguration (NEW) (EXPERIMENTAL) (NEW)
[ ]   IP: tunneling (NEW) (EXPERIMENTAL) (NEW)
[ ]   IP: GRE tunnels over IP (NEW) (EXPERIMENTAL) (NEW)
[ ]   IP: ARP daemon support (EXPERIMENTAL) (NEW) (EXPERIMENTAL)
[ ]   IP: TCP Explicit Congestion Notification support (NEW)
[ ]   IP: TCP syncookie support (disabled per default) (NEW)
[ ]   The IPv6 protocol (EXPERIMENTAL) (NEW)
[ ]   Kernel httpd acceleration (EXPERIMENTAL) (NEW)
[ ] Asynchronous Transfer Mode (ATM) (EXPERIMENTAL)
[ ] 802.1Q VLAN Support (EXPERIMENTAL)
---
[ ] The IPX protocol Layer (EXPERIMENTAL)
v(+)
```

<Select> < Exit > < Help >

Linux Kernel v2.4.17-uc1 Configuration

Network device support

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in [] excluded <M> module < > module capable

```
[*] Network device support?
ARCnet devices --->
[ ] Dummy net driver support (NEW)
[ ] Bonding driver support (NEW)
[ ] EQL (serial line load balancing) support (NEW)
[ ] Universal TUN/TAP device driver support (NEW)
[ ] Ethertap network tap (OBSOLETE) (NEW)
Ethernet (10 or 100Mbit) --->
Ethernet (1000 Mbit) --->
[ ] FDDI driver support (NEW)
[ ] HIPPI driver support (EXPERIMENTAL) (NEW)
[*] PPP (point-to-point protocol) support (NEW)
[*] PPP multilink support (EXPERIMENTAL) (NEW)
[*] PPP support for async serial ports (NEW)
[ ] PPP support for sync tty ports (NEW)
[ ] PPP Deflate compression (NEW)
[ ] PPP BSD-Compress compression (NEW)
[ ] PPP over Ethernet (EXPERIMENTAL) (NEW)
[ ] SLIP (serial line) support (NEW)
Wireless LAN (non-hamradio) --->
v(+)
```

<Select> < Exit > < Help >

Linux Kernel v2.4.17-uc1 Configuration

Block devices

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in
[] excluded <M> module < > module capable

```
[ ] Normal PC floppy disk support
[ ] XT hard disk support
[ ] Compaq SMART2 support
[ ] Compaq Smart Array 5xxx support
[ ] Mylex DAC960/DAC1100 PCI RAID Controller support
[*] Loopback device support
[ ] Network block device support
[*] RAM disk support
(4096) Default RAM disk size (NEW)
[ ] Initial RAM disk (initrd) support
[*] ROM disk memory block device (blkmem)
(AMD) FLASH type
(AUTO) FLASH size
(AUTO) FLASH bit width
```

<Select> < Exit > < Help >

Linux Kernel v2.4.17-uc1 Configuration

File systems

Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help. Legend: [*] built-in
[] excluded <M> module < > module capable

```
^(-)
[ ] Ext3 journalling file system support (EXPERIMENTAL)
[ ] DOS FAT fs support
[ ] EFS file system support (read only) (EXPERIMENTAL)
[ ] Journalling Flash File System (JFFS) support
[ ] Journalling Flash File System v2 (JFFS2) support
[ ] Compressed ROM file system support
[ ] Virtual memory file system support (former shm fs)
[*] Simple RAM-based file system support
[ ] ISO 9660 CDROM file system support
[ ] Minix fs support
[ ] FreeUxFS file system support (VERITAS UxFS(TM) compatible)
[ ] NTFS file system support (read only)
[ ] OS/2 HPFS file system support
[*] /proc file system support
[ ] /dev file system support (EXPERIMENTAL)
[ ] QNX4 file system support (read only) (EXPERIMENTAL)
[*] ROM file system support
[*] Second extended fs support
[ ] System U/Xenix/U7/Coherent file system support
[ ] UDF file system support (read only)
v(+)
```

<Select> < Exit > < Help >

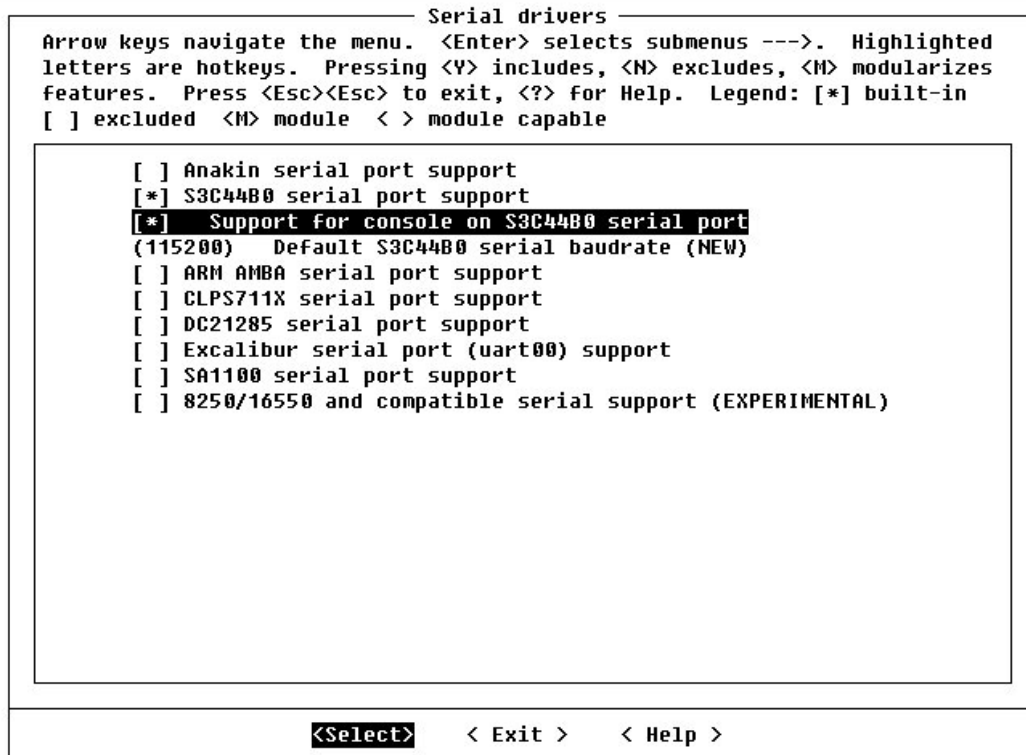


그림 14 커널옵션 설정

옵션 설정이 완료 되면 설정을 저장하고 나와서 컴파일 해준다.

➤ make dep; make clean; make zImage

중간에 에러 없이 컴파일이 되었으면 ~/arch/armnommu/boot 에 zImage라는 압축된 커널이미지 파일이 생성되었을 것이다.

4.3 Target 보드에 저장

이제 생성된 커널이미지를 BLOB을 통해 Target 보드에 다운로드 해보자. BLOB을 이용해 커널이미지를 Flash Memory에 저장하기 위해서는 먼저 시리얼 포트를 통해 Host PC에 저장된 커널이미지를 보드의 RAM에 다운로드해 주어야한다. 그리고 Flash memory의 커널이미지 블록을 지운다음 RAM에 저장된 이미지를 복사해

주어해 한다.

우선 Chapter3의 부트로더가 정상적으로 올라가 있다는 가정하에 시작한다. 보드의 전원을 넣고 부트 메시지가 나오면 아무키나 눌러 커널을 부트시키지 말고 BLOB명령 모드로 들어가자.

4.3.1 RAM 에 커널이미지 다운로드

그림 15와 같이 다운로드 명령어와 이미지이름을 입력하여 BLOB이 다운로드 대기상태로 들어가면 터미널 에뮬레이터 프로그램의 파일송신메뉴를 이용 시리얼포트를 통해 파일전송을 해준다. 이때 에뮬레이터의 사용프로토콜은 XPROTOCOL을 선택한다.

```
...  
blob> xdownload kernel5  
Start sending using the XMODEM protocol (repeated ^X to quit).  
C
```

그림 15 다운로드 대기 화면

4.3.2 다운로드한 이미지를 Flash 에 저장

다운로드를 성공적으로 끝마쳤으면 그림 16과 같이 flash 명령과 다운로드한 이미지 이름을 입력하면 플래쉬 메모리의 커널블록을 지우고 다운로드한 이미지를 저장한다.

```
blob> flash kernel  
Start erase block 00010000 [SUCCESS]  
Start erase block 00020000 [SUCCESS]  
Start erase block 00030000 [SUCCESS]  
Start erase block 00040000 [SUCCESS]  
Start erase block 00050000 [SUCCESS]  
Start erase block 00060000 [SUCCESS]  
Start erase block 00070000 [SUCCESS]  
Start erase block 00080000 [SUCCESS]
```

⁵ X 프로토콜은 전송중 에러 발생율이 높으므로 대기상태가 되자마자 빠른 손동작으로 파일을 전송해주어야 한다.

```

Write the Kernel Block...
00000000 00010000 00020000 00030000 00040000 00050000 00060000 00070000
Saving h
blob>

```

그림 16 flash 저장 화면

이미지를 다운로드 시 저장되는 램의 영역과 플래쉬에 저장할때 사용되는 플래쉬의 주소 영역은 표2를 참조 바란다.

4.3.3 커널 부팅

램 영역에 LOAD되어 있는 커널 이미지 위치로 PC(Program Counter)를 옮겨 리눅스 커널을 부팅한다.

```

...
blob> boot
Starting kernel ...

Uncompressing Linux..... done, booting the kernel.
Linux version 2.4.17-uc1 (root@noscom) (gcc version 2.95.3 20010315 (release))(C3
Processor: Samsung S3C44B0 revision 1
Architecture: mba-44b0
On node 0 totalpages: 2048
zone(0): 0 pages.
...

```

그림 17 커널 부팅 화면

그림 17과 같이 커널압축을 풀고 Architecture 확인 메시지가 출력되면 이번 Chapter에서 원하는 수준까지는 성공한 셈이다. 아마도 루트파일시스템을 올리는 부분에서 에러메세지가 출력될것이다. 커널이 제대로 동작되는지는 확인 되었지만 아직 루트파일 시스템을 올리지 않았기에 커널이 루트파일 시스템을 찾다가 실패했기 때문이다. 이 다음 Chapter에서 롬디스크를 이용하여 루트파일 시스템을 올릴것이다.

Chapter 5 루트파일 시스템

리눅스에서 블록 디바이스는 문자 디바이스와 다르게 랜덤한 접근이 가능하여 원하는 위치에 데이터를 접근하여 쓰거나 지울 수 있다. 이러한 디바이스로 가까워서 볼 수 있는 것이 하드디스크나 플로피 디스크일것이다. 실제로 우리가 사용하는 리눅스 박스 그 자체는 커널만 가지고는 아무일도 할 수가 없다. 유저 영역에서 이 커널과 인터페이스 할수 있는 셸과 같은 시스템 응용 프로그램들이 포함되어져야만 한다. 그리고 리눅스의 경우에는 이를 포함하고 있는 파일 시스템을 커널 부팅의 마지막 과정에서 마운트하여 사용하게 된다. 앞의 Chapter에서 까지 실제로 커널 부팅이 거의 완료된것이고, 이제 남은 것은 파일시스템을 마운트하여 응용프로그램을 사용할수 있게 하는것이다. 일반 PC의 리눅스에서는 하드디스크를 마운트하여 사용하지만 임베디드 시스템에서는 대부분 하드디스크는 존재하지 않으므로 하드디스크의 역할을 램과 롬으로 대체할 수밖에 없다. 따라서 여기서는 Flash Rom과 Dram을 블록 디바이스로 만들어 하드디스크 처럼 사용할것이다. Flash Rom 은 롬파일시스템을 사용하여 루트파일시스템을 Read-Only로 사용하게 되고 항상 가변하는 /proc이나 /temp, /var 같은 디렉토리들은 램디스크를 사용하게 될 것이다. 그리고 나중에 유저 디스크영역으로 마운트하여 하드디스크처럼 맘대로 Read/Write 할수 있는 JFFS(Journaling Flash File System)을 구축할것이다.

5.1 블록 디바이스 수정

blkmem.c 에서 BLKMEM_MAJOR 값을 30으로 수정하고 플래쉬의 루트파일시스템이 저장될 주소를 정해준다. 31은 후에 jffs에 사용 할것이다.

여기서 0x00090000은 flash의 메모리맵과 관련있다. 부트로더에서 플래쉬영역을 3개로 나누어 저장하였으며, 각각 blob은 (0x0 ~ 0xFFFF), kernel은 (0x10000 ~ 0x0008FFF) romdisk 영역은 (0x00090000 ~ 0x001FFFFF) 까지이다. 그러므로 romdisk로 사용할 영역의 시작 주소 0x00090000으로 설정해주는 것이다.

➤ vi ~/drivers/block/blkmem.c

```
...  
#define FALSE (0)  
  
#define BLKMEM_MAJOR 30
```

```

#define MAJOR_NR BLKMEM_MAJOR
...

...

} arena[] = {

#ifdef CONFIG_ARCH_S3C44B0
    {0, ((int) (0x00090000)), -1},
#endif

#ifdef INTERNAL_ROMARRAY
...

```

코드 11 block device driver 수정

코드 11과 같이 수정을 해주고 커널에서의 파일시스템과 블록디바이스 옵션은 그림 14 에서와 같이 설정해주어야 한다.

그리고는 커널소스가 수정되었으므로 다시 컴파일 해주어야 할것이다.

```
# make dep; make clean; make zImage
```

그리고는 BLOB을 이용하여 생성된 커널이미지를 다시 Target 보드에 저장하여 준다. 만일 롬디스크 이미지가 플래쉬 롬에 올려져 있다면 롬디스크의 루트파일시스템이 마운트되고 리눅스 프롬프트가 보일것이다. 하지만 아직 롬디스크 이미지를 마운트하지 않았으므로 전과 같은 어레 메시지가 출력될것이다. 자 이제 롬디스크를 올려 보자.

5.2 롬디스크

롬디스크를 직접 모두 작성해보는 것이 좋겠으나 앞서 언급한대로 이번에는 최대한 빠르고 쉽게 만들기 위해 이미 만들어져 있는 디스크이미지를 이용하도록 한다. 롬디스크는 일단 마운트되어 루트파일시스템이 구축되면 파일을 읽기밖에 할수없다. 추가, 삭제가 불가능한것이다. 그러므로 원하는 파일의 추가,삭제를 원할 경우 다음과 같이 이미지를 Host PC상에서 수정하여 다시 롬디스크이미지로 만들어

Target보드에 다운로드하여 사용하여야 한다.

- ① 롬 디스크를 만들기 위한 디렉토리 생성

```
# mkdir romfs_tmp
```

- ② 새로운 롬 파일 시스템을 만들기 위해 롬 디스크 이미지를 루프 디바이스 (loop device)로 변환

```
# losetup /dev/loop0 romdisk.img
```

- ③ romfs_tmp 디렉토리에 변환된 루프 디바이스를 마운트

```
# mount -t romfs /dev/loop0 romfs_tmp
```

이제 단말기에서 사용할 루트파일 시스템의 디렉토리 구조를 romfs_tmp 에서 확인할 수 있다.

- ④ 앞에서 만든 롬 디스크를 압축하여 romfs.tgz 파일 압축

```
# tar cvfz romfs.tgz romfs_tmp
```

- ⑤ romfs_tmp 언마운트

```
# umount romfs_tmp
```

- ⑥ 루프디바이스를 닫고, romfs_tmp 디렉토리를 삭제

```
# losetup -d /dev/loop0  
# rmdir romfs_tmp
```

- ⑦ 앞에서 만든 롬 디스크의 압축을 해제

```
# tar xvfz romfs.tgz
```

제대로 압축이 풀리면 romfs_tmp 가 다시 생성되어 있을 것이다.

- ⑧ 압축된 롬 디스크 압축 파일 삭제

```
# rm romfs.tgz
```

- ⑨ 생성된 romfs_tmp를 romfs_disk로 이름 변경

```
# mv romfs_tmp romfs_disk
```

- ⑩ 이 romfs_disk 디렉토리가 Target 보드에서 마운트하여 사용하게 된 루트파일 시스템 그 자체이다. 직접 이곳에서 원하는 파일을 추가, 삭제하여 주고 다음 과정에 따라 다시 보드에 다운로드해주면 되는 것이다.

- ⑪ 리눅스 명령어 genromfs를 이용하여 생성한 romfs_disk 디렉토리 전체를 타겟에서 사용할 롬파일시스템이미지로 변환

```
# genromfs -v -V "ROMdisk" -f romdisk.new -d romfs_disk > romdisk.map
```

(개발 과정에서 자주 사용하므로 셸스크립트로 만들어 놓고 쓰는 것이 좋다)

실행이 완료되면 romdisk.new 라는 파일이 생성된다. 생성된 롬 디스크를 Target 보드에서 사용하기 위해 부트로더를 이용해 단말기에 다운로드하고 다시 커널을 부트 시킨다면

```
blob> boot
```

```
Starting kernel ...
```

```
Uncompressing Linux..... done, booting the kernel.
```

```
Linux version 2.4.17-uc1 (root@noscom) (gcc version 2.95.3 20010315 (release))(C3
```

```
Processor: Samsung S3C44B0 revision 1
```

```
Architecture: mba-44b0
```

```
On node 0 totalpages: 2048
```

```
zone(0): 0 pages.
```

```
zone(1): 2048 pages.
```

```
zone(2): 0 pages.
Kernel command line: root=/dev/ram0
Calibrating delay loop... 30.82 BogoMIPS
Memory: 8MB = 8MB total
Memory: 6988KB available (848K code, 169K data, 40K init)
Dentry-cache hash table entries: 1024 (order: 1, 8192 bytes)
Inode-cache hash table entries: 512 (order: 0, 4096 bytes)
Mount-cache hash table entries: 512 (order: 0, 4096 bytes)
Buffer-cache hash table entries: 1024 (order: 0, 4096 bytes)
Page-cache hash table entries: 2048 (order: 1, 8192 bytes)
POSIX conformance testing by UNIFIX
Linux NET4.0 for Linux 2.4
Based upon Swansea University Computer Society NET3.039
Initializing RT netlink socket
Starting kswapd
JFFS version 1.0, (C) 1999, 2000 Axis Communications AB
ttyS0 at I/O 0x1d00000 (irq = 3) is a S3C44B0
ttyS1 at I/O 0x1d04000 (irq = 2) is a S3C44B0
block: 64 slots per queue, batch=16
RAMDISK driver initialized: 16 RAM disks of 4096K size 1024 blocksize
Blkmem copyright 1998,1999 D. Jeff Dionne
Blkmem copyright 1998 Kenneth Albanowski
Blkmem 1 disk images:
0: 90000-190FFF [VIRTUAL 90000-190FFF] (RO)
loop: loaded (max 8 devices)
PPP generic driver version 2.4.1
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 512 bind 512)
VFS: Mounted root (romfs filesystem) readonly.
Freeing init memory: 40K
Thu Jun 06 17:49:00 ??? 2002
Expanding RAM File System into /dev/ram0 and /dev/ram1...
Mounting filesystems
```

```
uClinux kernel-2.4.17
www.NOSCOM.co.kr
Porting by Seongmin Ko

SIOCSIFADDR: No such device
SIOCADDRT: No such device

Sash command shell (version 1.1.1)
/>
```

그림 18 루트파일 시스템이 마운트된 후

롬디스크이미지는 여러군데서 구할수 있다. 특히 uClinux.org 에서 제공해주는 uClinux source distribution package 에는 커널과 라이브러리, 응용 코드를 모두 담고 있어 사용하기 가장 편하다. 옵션에서 원하는 응용프로그램을 선택만 해주면 컴파일 되어 롬디스크이미지로 만들수 있기 때문이다.

여기까지 우리가 실제로 리눅스를 사용할수 있는 기본 시스템이 갖추어졌다. 이제 실행을 원하는 응용프로그램을 위의 과정을 따라 롬디스크에 추가해주면 Target 보드에서 실행할수 있게 된것이다.

그럼 이제부터는 Target 보드의 주변장치를 사용할수 있게 하는 디바이스 드라이버를 작성해 본다.

Chapter 6 디바이스 드라이버

디바이스 드라이버는 커널과 유저 사이에서 하드웨어 장치를 직접 제어하는 코드로 프로그래머로 하여금 하드웨어에 비의존적인 응용프로그램을 작성할수 있는 인터페이스를 제공해주게 된다. 즉 디바이스 드라이버를 사용하면 입출력 장치의 파일 조작 함수를 사용하여 하드웨어를 조작할수 있게 되는 것이다. 리눅스에서는 모든 하드웨어 장치를 파일시스템처럼 보이게 추상화하므로 이와 같이 표준 시스템 콜과 같은 함수를 이용하여 open, close, read, write 등의 명령을 이용하는 것이 가능하다. 또한 디바이스드라이버는 /dev 에 그 디바이스를 연결시켜 주는 노드를 통해 실질적인 사용이 가능하게 된다. 디바이스 드라이버는 이 노드의 major 번호에 의해 구분되며 각각의 소자는 minor 번호에 의하여 연결된다.

디바이스 드라이버는 크게 character device와 block device, network device 로 나뉘 수 있다. 이중 상당수의 디바이스가 character device이다.

6.1 LRZSZ

lrzsz은 XMODEM, YMODEM, ZMODEM 파일 전송 프로토콜을 제공하는 유닉스 통신 패키지이다.

Host PC에서 크로스컴파일한 응용프로그램을 Target 보드에서 실행하기 위해서는 응용프로그램 파일을 타겟보드에 다운로드해주어야한다. 아직까지의 과정에서는 뽀족한 방법없이 롬디스크이미지에 응용프로그램을 포함시켜 다시 이미지화한 다음 BLOB으로 다시 다운로드 하는 방법밖에 없다. 그러나 이 방법은 파일하나를 추가하려고 하기에는 너무나 성가시고 오랜 시간이 걸린다. 그래서 필요한 프로그램이 Target 보드를 모니터하고 있는 시리얼포트를 통해 직접 다운로드할수 있게 해주는 것이 lrzsz 이다.

lrzsz는 MINICOM같은 프로그램에서도 파일전송을 위해 사용하고 있으며 X, Y, Z 모뎀 프로토콜을 이용하여 파일 송수신이 가능하다. 여기서는 이프로그램을 크로스 컴파일하여 루트파일 시스템에 추가 시키고 디바이스 드라이버를 테스트하는 응용 프로그램을 보드로 다운로드 할때마다 사용할것이다.

6.1.1 소스 컴파일

- ① lrzsz 소스 다운받기
- ② <http://www.ohse.de/uwe/releases/lrzsz-0.12.20.tar.gz> 에서 lrzsz 소스를 다운 받아 압축을 푼다.

```
# tar zxvf lrzsz-0.12.20.tar.gz
```

③ configuration

```
# cd lrzsz-0.12.20
# CC=arm-elf-gcc OBJCOPY=arm-elf-objcopy RANLIB=arm-elf-ranlib
AR=arm-elf-ar NM=arm-elf-nm LDFLAGS=-elf2flt ./configure --with-
linux-prefix=/root/Nsc4kl/kernel/uClinux-2.4.x
```

④ compile

```
# make
```

성공적으로 컴파일이 되었으면 ~src 디렉토리에 lrz와 lsz 파일이 생성되었을 것이다.

⑤ romdisk 에 추가

생성된 lrz 파일을 롬 디스크의 /usr/bin 디렉토리에 복사해 넣고 다시 롬디스크이미지를 만들어 보드에 다운로드하여 저장한다.

6.1.2 테스트

Lrzsz 가 동작하기 위해서는 커널 옵션 general setup의 System V IPC와 NWFPE math emulation이 체크 되어 컴파일했는지 반드시 확인해본다

lrzsz를 추가해준 롬디스크이미지를 새로 다운로드한후 커널을 부팅시켜 램디스크 디렉토리인 /user 로 옮겨 가서 lrz를 실행시킨다.

```
# /user> lrz
```

그리고는 다운로드 대기 모드로 들어가면 통신 에뮬레이터의 파일송신메뉴를 선택하여 송신 파일을 선택하여 파일을 송신한다. (미니컴의 경우 Ctrl-A-s를 눌러 Upload 프로토콜 선택 메뉴에서 원하는 전송 프로토콜을 선택하면 파일 선택 창이 나온다. 여기서 전송할 파일을 스페이스바를 눌러 선택 한후 엔터를 눌러 전송한다.)

현재 디렉토리에 송신한 파일이 수신 되었으면 잘 컴파일된것이다. 수신한 파일이 실행파일일경우 반드시 파일모드를 실행모드로 바꿔주어야 한다. 수신시 실행모드

가 무조건 바뀐다.

```
# chmod +x 수신 파일이름
```

이제 앞으로 디바이스 드라이버를 작성한후 위의 방법으로 보드에 테스트 응용 프로그램을 송신하여 디바이스 드라이버를 테스트하게 될것이다. 그외에 모든 응용 프로그램은 이런식으로 다운로드하여 사용할수 있게 된다.

6.2 MTD & JFFS

리눅스에서 MTD(Memory Technology Device)는 플래쉬나 램과 같은 메모리들을 쉽게 읽고 쓰고, 지울수 있도록 한 일반적인 인터페이스이다. MTD 블록 디바이스는 JFFS, JFFS2 그리고 CRAMFS 파일 시스템으로 마운트 될 수 있고 Intel, sharp, AMD 그리고 Fujitsu와 같은 업체에서 공동으로 사용하는 CFI(Common Flash Interface)를 지원함으로써 NOR 플래쉬를 폭넓게 지원한다. 이것을 통해 플래쉬의 버스폭이나 버스폭을 구현하기 위한 칩의 개수가 설정 되어지거나 자동적으로 찾아질수 있다. MTD 드라이버 층은 또한 하나의 플래시에서 여러 플래시 파티션을 제공한다.

앞서 언급했듯이 MTD 드라이버를 접급하기 위한 파일 시스템에는 JFFS, JFFS2, CRAMFS가 있는데 여기서는 JFFS와 JFFS2에 대해서만 다룬다. JFFS(Journaing Flash File System)은 스웨덴의 Axis Communications에 의해 개발된 파일 시스템으로 데이터와 메타데이터를 포함하는 노드는 저장 가능한 공간에 연속적으로 플래쉬에 저장되어 파일의 읽고 쓰기가 모두 가능한 파일시스템이다.

Target 보드에 내장된 플래쉬는 Hynix 의 HY29LV160BT로서 AMD의 16비트 플래쉬와 완벽히 호환되는 칩으로 CFI를 지원하는 NOR 플래쉬이다. 이미 리눅스 커널에서 CFI를 지원하는 플래쉬 오퍼레이션을 담당하는 부분을 구현 해놓았기 때문에, 보드에 맞는 MTD 드라이버를 쉽게 작성 할수 있다. 실제로 플래쉬 메모리를 파티션하고 이미 구현되어 있는 오퍼레이션을 연결해주면 작업이 완성된다.

6.2.1 드라이버 작성

드라이버소스 nsc4kl.c를 ~/drivers/mtd/maps 에 복사해준다.

이파일은 MTD 드라이브를 사용해 NSC4000L의 플래쉬정보를 등록하고 파티션을

나눠 주도록 플래시메모리의 주소와 크기 그리고 partition map를 설정해주는 역할을 한다.

```
static struct mtd_info *mymtd;

int __init init_nsc4kl(void)
{
    printk(KERN_NOTICE "NSC4000L flash device: %x at %x\n",
           FLASH_SIZE, FLASH_PHYS_ADDR);

    nsc4kl_map.map_priv_1 = (unsigned long)ioremap(FLASH_PHYS_ADDR,
           FLASH_SIZE);

    if (!nsc4kl_map.map_priv_1) {
        printk("Failed to ioremap for nsc4kl Flash\n");
        return -EIO;
    }

    mymtd = do_map_probe("cfi_probe", &nsc4kl_map);
    if (!mymtd)
        return -ENXIO;

    mymtd->module = THIS_MODULE;

    printk(KERN_NOTICE "NSC4000L flash device initialized\n");

    add_mtd_partitions(mymtd, nsc4kl_parts, PARTITION_COUNT);

    return 0;
}

static void __exit cleanup_nsc4kl(void)
{
    if (mymtd) {
        del_mtd_partitions(mymtd);
        map_destroy(mymtd);
    }
}
```

```

    if (nsc4kl_map.map_priv_1) {
        iounmap((void *)nsc4kl_map.map_priv_1);
        nsc4kl_map.map_priv_1 = 0;
    }
}
module_init(init_nsc4kl);
module_exit(cleanup_nsc4kl);

```

코드 12 MTD 드라이버 초기화와 종료

초기화와 종료는 `init_nsc4kl()` 함수와 `cleanup_nsc4kl()`이 해주어 링크시에 `__init` 섹션에 `init_nsc4kl()` 함수를 위치시키기 때문에 부팅시 호출되어 초기화 된다. 이때 이루어지는 작업은 플래쉬 메모리의 물리적 주소가 유효한지 검사한 후, `do_map_probe()` 함수를 통해 “`cfi_probe`” 드라이버를 찾아 플래쉬 오퍼레이션을 등록한 `map_info` 구조체를 연결한다. 그리고 끝으로 `add_mtd_partitions()`를 통해 플래쉬의 파티션을 생성한다. 이때 파티션 정보를 담고 있는 `mtd_partition` 구조체인 `nsc4kl_parts`를 참조하게 된다.

```

__u8 nsc4kl_read8(struct map_info *map, unsigned long ofs)
{
    return __raw_readb(map->map_priv_1 + ofs);
}

__u16 nsc4kl_read16(struct map_info *map, unsigned long ofs)
{
    return __raw_readw(map->map_priv_1 + ofs);
}

__u32 nsc4kl_read32(struct map_info *map, unsigned long ofs)
{
    return __raw_readl(map->map_priv_1 + ofs);
}

void nsc4kl_copy_from(struct map_info *map, void *to, unsigned long from, ssize_t len)
{

```

```

        memcpy(to, (void*)(map->map_priv_1 + from), len);
    }

void nsc4kl_write8(struct map_info *map, __u8 d, unsigned long adr)
{
    __raw_writeb(d, map->map_priv_1 + adr);
    mb();
}

void nsc4kl_write16(struct map_info *map, __u16 d, unsigned long adr)
{
    __raw_writew(d, map->map_priv_1 + adr);
    mb();
}

void nsc4kl_write32(struct map_info *map, __u32 d, unsigned long adr)
{
    __raw_writel(d, map->map_priv_1 + adr);
    mb();
}

void nsc4kl_copy_to(struct map_info *map, unsigned long to, const void *from,
                    ssize_t len)
{
    switch (map->buswidth) {
        case 4:                // 32 bits
            while (len > 0) {
                __u32 d = *((__u32*)from)++;
                nsc4kl_write32(map, d, to);
                to += 4;
                len -= 4;
            }
            break;
        case 2:                // 16 bits
            while (len > 0) {
                __u16 d = *((__u16*)from)++;

```

```

        nsc4kl_write16(map, d, to);
        to += 2;
        len -= 2;
    }
    break;
case 1:                // 8 bits
    while (len > 0) {
        __u8 d = *((__u8*)from)++;
        nsc4kl_write8(map, d, to);
        to++;
        len--;
    }
    break;
}
}

/*****/
/* first flash memory map_info */
/*****/

static struct mtd_info *mymtd;

int __init init_nsc4kl(void)
{
    printk(KERN_NOTICE "NSC4000L flash device: %x at %x\n",
           FLASH_SIZE, FLASH_PHYS_ADDR);

    nsc4kl_map.map_priv_1 = (unsigned long)ioremap(FLASH_PHYS_ADDR,
           FLASH_SIZE);

    if (!nsc4kl_map.map_priv_1) {
        printk("Failed to ioremap for nsc4kl Flash\n");
        return -EIO;
    }

    mymtd = do_map_probe("cfi_probe", &nsc4kl_map);
    if (!mymtd)

```

```

        return -ENXIO;

    mymtd->module = THIS_MODULE;

    printk(KERN_NOTICE "NSC4000L flash device initialized\n");

    add_mtd_partitions(mymtd, nsc4kl_parts, PARTITION_COUNT);

    return 0;
}

static void __exit cleanup_nsc4kl(void)
{
    if (mymtd) {
        del_mtd_partitions(mymtd);
        map_destroy(mymtd);
    }
    if (nsc4kl_map.map_priv_1) {
        iounmap((void *)nsc4kl_map.map_priv_1);
        nsc4kl_map.map_priv_1 = 0;
    }
}

```

코드 13 nsc4kl_map 과 read,write operation

Map_info 구조체인 nsc4kl0_map에 읽기, 쓰기 오퍼레이션을 등록하였다. 버스크 기마다 각각의 읽기 쓰기 함수를 구현하였으나, 실제로 함수의 내용은 미리 구현되어 있는 __raw_read와 __raw_write를 이용한 것을 알수 있다.

끝으로 파티션 정보를 갖고 있는 부분을 살펴 보겠다.

```

#define FLASH_PHYS_ADDR    0x02000000
#define FLASH_SIZE         0x00200000

#define FLASH_BLOCK1_BASE  0x00000000
#define FLASH_KERNEL_SIZE  0x00100000

```



```

...

/*****
/* first flash memory partition info */
*****/

struct mtd_partition nsc4kl_parts[] = {
    {
        name:          "User storage 1",
        size:           FLASH_KERNEL_SIZE, // part size
        offset: FLASH_BLOCK1_BASE          // part start offset
    },
    {
        name:          "User storage 2",
        size:           MTDPART_SIZ_FULL, // rest size
        offset: MTDPART_OFS_APPEND        // prepart end
    }
};

```

코드 14 Partition 정보

Mtd_partition 구조체에서 name 멤버 변수는 파티션의 이름을 나타내고, size 멤버 변수는 파티션의 크기를 나타낸다. 그리고 offset 멤버 변수는 해당 파티션의 시작 위치를 나타낸다. 여기에서는 쓰이지 않았지만, 파티션에 대한 읽기-쓰기 또는 읽기 전용등과 같이 설정을 할수 있는 mask_flags 멤버변수가 있다.

여기서 사용할 수 있는 매크로들이 몇 가지 있다. MTDPART_OFS_APPEND라는 매크로는 이전 파티션의 끝을 나타내며, offset 멤버변수에서 쓰일수 있다. MTDPART_SIZ_FULL은 플래쉬의 끝까지 파티션을 잡는 것으로, size 멤버변수에서 쓰인다. MTD_WRITEABLE은 mask_flag 멤버변수에서 쓰이는 것으로 파티션은 읽기 전용으로 만든다.

NSC4000L의 경우 2개의 플래쉬중 하나는 kernel, blob, romdisk 이미지를 저장하고 있어 나머지 하나를 유저영역으로 사용할수 있도록 MTD를 만들어 주었다.

6.2.2 옵션메뉴스크립트 추가

위에서와 같이 커널소스상에 MTD 디바이스 드라이버 코드를 추가 시켜주었으면 커널 옵션 설정메뉴에서 MTD 드라이버를 선택할수 있도록 스크립트에 메뉴를 추

가해주어 커널 컴파일시 같이 컴파일 될수 있도록 한다. 우선 커널 옵션 설정시 선택할수 있도록 커널의 ~/drivers/mtd/maps/Config.in을 코드 15와 같이 3줄의 코드를 삽입해 준다.

```
...  
int '    Bus width in octets' CONFIG_MTD_PHYSMAP_BUSWIDTH 2  
fi  
  
dep_tristate '    CFI Flash device mapped on Noscom NSC4KL'  
CONFIG_MTD_NSC4KL $CONFIG_MTD_CFI $CONFIG_MTD_PARTITIONS  
  
if [ "$CONFIG_MTD_NSC4KL" = "y" ]; then  
    bool '    Use a JFFS for Root Filesystem' CONFIG_MTD_JFFS_ROOT  
fi  
  
dep_tristate '    CFI Flash device mapped on Dragonix VZ'  
CONFIG_MTD_DRAGONIX                                $CONFIG_MTD_CFI_INTELEXT  
$CONFIG_MTD_CFI_AMDSTD $CONFIG_MTD_PARTITIONS  
...
```

코드 15 ~drivers/mtd/maps/Config.in 수정

그리고 옵션이 설정되면 컴파일시에 MTD드라이버가 함께 컴파일 할 수 있도록 ~/drivers/mtd/maps/Makefile 을 수정한다.

```
...  
obj-$(CONFIG_MTD_KeyTechnology) += keyTechnology.o  
obj-$(CONFIG_MTD_COBRA5272) += cobra5272.o  
obj-$(CONFIG_MTD_NSC4KL) += nsc4kl.o  
  
include $(TOPDIR)/Rules.make
```

코드 16 ~drivers/mtd/maps/Makefile 수정

다시 커널을 컴파일 해주어야 한다. 하지만 커널 옵션 설정메뉴에는 MTD선택 메뉴가 아예 없을 것이다.

~/arch/armnommu/config.in을 보면 커널옵션중 CONFIG_ALIGNMENT_TRAP이

설정 되어 있을 경우에만 설정 메뉴가 나타나게 되어 있는데 alignment_trap 옵션을 설정하고 컴파일 하면 에러가 발생하므로 일단은 주석으로 조건문을 막아준다.

```
...  
#if [ "$CONFIG_ALIGNMENT_TRAP" = "y" ]; then  
    source drivers/mtd/Config.in  
#fi  
...
```

코드 17 ~/arch/armnommu/config.in 의 수정

그리고 다시

➤ make menuconfig

그림19와 같이 MTD 드라이버 옵션을 정해준다.

```

----- Memory Technology Devices (MTD) -----
Arrow keys navigate the menu. <Enter> selects submenus --->.
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes,
<M> modularizes features. Press <Esc><Esc> to exit, <?> for Help.
Legend: [*] built-in [ ] excluded <M> module < > module capable

[*] Memory Technology Device (MTD) support
[ ] Debugging
[*] MTD partitioning support
[ ] RedBoot partition table parsing
[ ] Compaq bootldr partition table parsing
[ ] ARM Firmware Suite partition parsing
--- User Modules And Translation Layers
[*] Direct char device access to MTD devices
[*] Caching block device access to MTD devices
[ ] FTL (Flash Translation Layer) support
[ ] NFTL (NAND Flash Translation Layer) support
RAM/ROM/Flash chip drivers --->
Mapping drivers for chip access --->
Self-contained MTD device drivers --->
v(+)

```

<Select> < Exit > < Help >

```

----- RAM/ROM/Flash chip drivers -----
Arrow keys navigate the menu. <Enter> selects submenus --->.
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes,
<M> modularizes features. Press <Esc><Esc> to exit, <?> for Help.
Legend: [*] built-in [ ] excluded <M> module < > module capable

[*] Detect flash chips by Common Flash Interface (CFI) probe
[ ] Detect non-CFI AMD/JEDEC-compatible flash chips
[ ] Flash chip driver advanced configuration options
[ ] Support for Intel/Sharp flash chips
[*] Support for AMD/Fujitsu flash chips
[ ] Support for RAM chips in bus mapping
[ ] Support for ROM chips in bus mapping
[ ] Support for absent chips in bus mapping
[ ] Older (theoretically obsoleted now) drivers for non-CFI chips

```

<Select> < Exit > < Help >

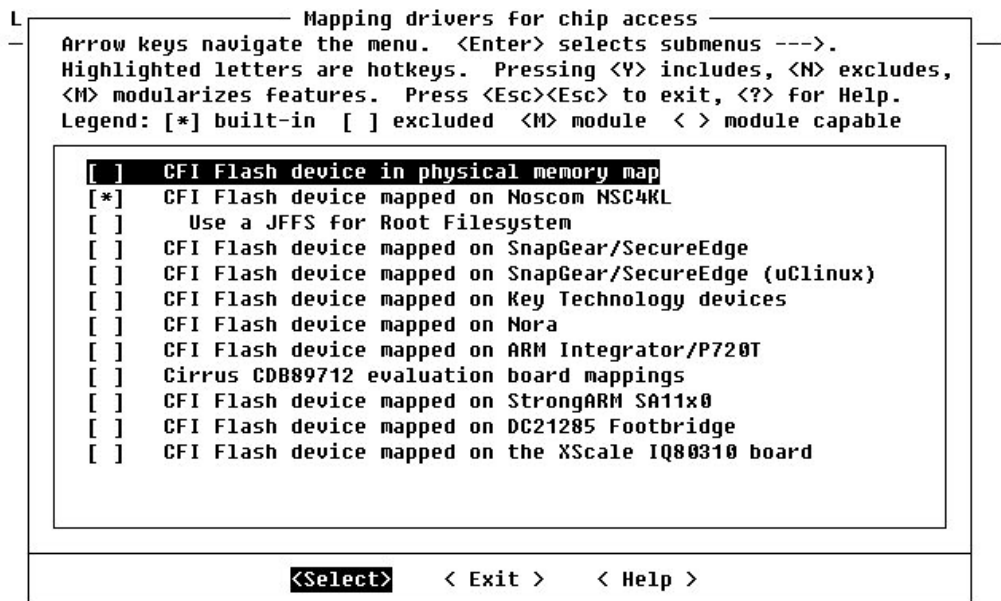


그림 19 MTD driver option select

그리고는 MTD파티션에는 파일 시스템을 JFFS로 정하고 그림 20과 같이 역시 커널에서 옵션을 설정해주어야 한다. 만일 JFFS2로 쓰고 싶다면 역시 옵션에서 체크해주기만 하면된다.

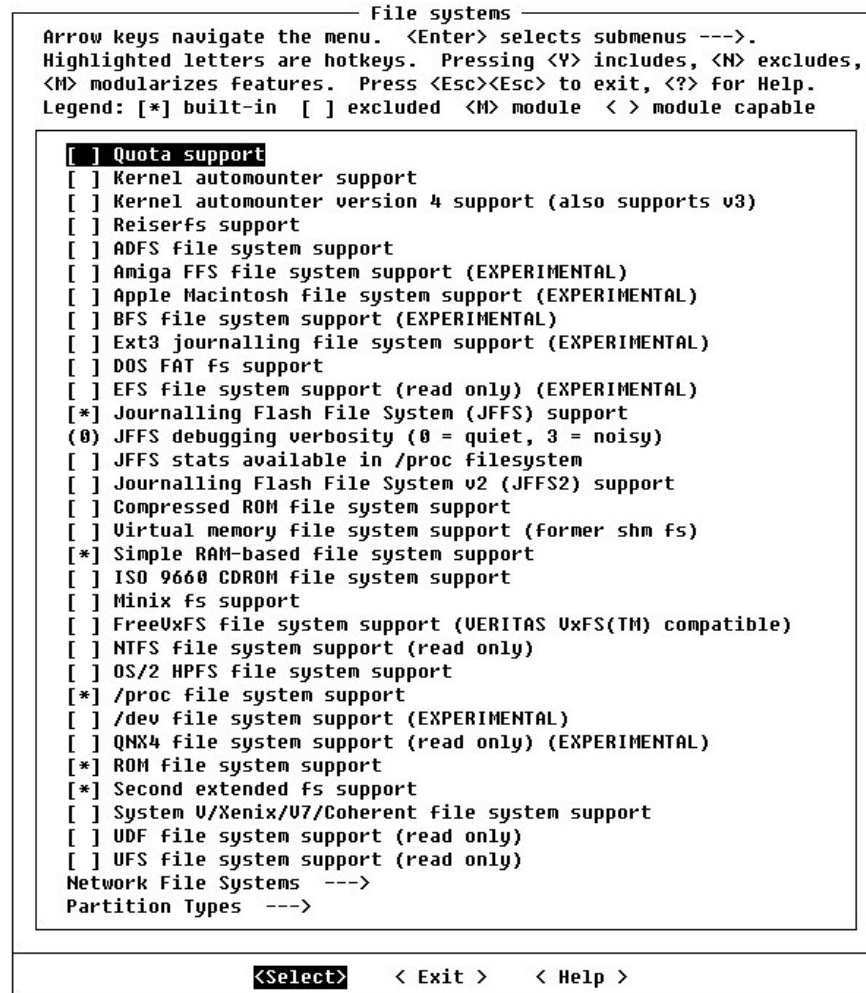


그림 20 JFFS 선택

이와 같이 설정후 컴파일을 하여 이상없이 컴파일 되면 다시 커널이미지를 Target 보드에 다운로드 하여 저장하고 부트시킨다. 제대로 되었으면 그림21 과 같이 리눅스 shell prompt를 볼수 있을 것이다. 만일 JFFS2를 선택했다면 커널 컴파일시 super.c 라는 파일이 존재하지않는다는 메시지가 출력되고 컴파일이 멈출것이다. super.c 를 uClinux distribution package의 커널소스에서 복사해 ~fs/jffs2 디렉토리에 추가시켜 다시 컴파일해주면 이상이 없을 것이다.

드라이버가 제대로 올라갔으면 다음과 같이 MTD block 0,1을 지워주고 jffs로 마운트 해준다.

```
# /> eraseall /dev/mtd0
Erased 1024 Kibyte @ 0 -- 100% complete.
# /> eraseall /dev/mtd1
Erased 1024 Kibyte @ 0 -- 100% complete.

# mount -f jffs /dev/mtdblock0 /mnt/flash0
# mount -f jffs /dev/mtdblock1 /mnt/flash1
```

```
IP Protocols: ICMP, UDP, TCP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 512 bind 512)
JFFS: Trying to mount a non-mtd device.
VFS: Mounted root (romfs filesystem) readonly.
Freeing init memory: 40K
Thu Jun 06 17:49:00 ??? 2002
Expanding RAM File System into /dev/ram0 and /dev/ram1...
Mounting filesystems

N  N  000  SSS  CCC  000  M  M  L  III  N  N  U  U  X  X
NN N  0  0  S  S  C  C  0  0  M  M  M  L  I  NN N  U  U  X  X
N N N  0  0  S  C  0  0  M  M  M  L  I  N N N  U  U  X  X
N NN  0  0  SSS  C  0  0  M  M  M  L  I  N  NN  U  U  X
N  N  0  0  S  C  0  0  M  M  M  L  I  N  N  U  U  X  X
N  N  0  0  S  S  C  C  0  0  M  M  L  I  N  N  U  U  X  X
N  N  000  SSS  CCC  000  M  M  LLLLL III N  N  UUU  X  X

uClinux kernel-2.4.17
www.NOSCOM.co.kr
Porting by Seongmin Ko

Sash command shell (version 1.1.1)
/> _
```

그림 21 부팅완료후의 리눅스 화면

6.2.3 테스트

df, mount 명령등을 실행하여 그림22와 같이 확인해 볼수 있다..

```
...
/> df
```

Filesystem	1k-blocks	Used	Available	Use%	Mounted on
/dev/rom0	1380	1380	0	100%	/
/dev/ram0	2011	19	1992	1%	/var
/dev/ram1	1003	13	990	1%	/user
/dev/mtdblock0	768	492	276	64%	/mnt/flash0
/dev/mtdblock1	768	0	768	0%	/mnt/flash1

```

/> mount
/dev/rom0 on / type romfs (ro)
/dev/ram0 on /var type ext2 (rw)
/dev/ram1 on /user type ext2 (rw)
/proc on /proc type proc (rw)
/dev/mtdblock0 on /mnt/flash0 type jffs (rw)
/dev/mtdblock1 on /mnt/flash1 type jffs (rw)
/>
...

```

그림 22 df, mount 를 실행한 화면

마지막으로 테스트를 위해 /etc/fstab 을 /mnt/flash0 로 복사해본다.

```
# /> cp /etc/fstab /mnt/flash0
```

특별한 에러 메시지 없이 /mnt/flash0에 fstab 파일이 복사 되어 있으면 드라이버가 제대로 올라간 것이다.

또 다른 방법은 /mnt/flash0 로 이동후 이 디렉토리에서 6.1.2 에서와같이 lrz를 이용 직접 파일을 다운로드 받아 제대로 수신이 되었는지 직접 확인해보는 방법도 있다. 코드18과 같이 간단한 프로그램을 크로스 컴파일 하여 단말기에 다운로드하여 실행해보기로 하자.

```

#include <stdio.h>

int main()
{
    printf("Hello World!!\n");
    return 0;
}

```

코드 18 간단한 테스트 프로그램


```
# arm-elf-gcc -o -elf2flt hello_world hello.c
```

hello_world 라는 파일이 생성될것이다.

단말기를 켜고 prompt가 뜨면 유저디스크에 파일을 다운 받는다.

```
# /> cd /mnt/flash0
# /mnt/flash0> lrz
?z waiting to receive.**B000000023be50
```

이상태에서 다운로드 대기 모드로 들어가면 통신 에뮬레이터의 파일 전송 메뉴에서 zmodem protocol을 선택하고 전송파일 메뉴에서 hello_world를 선택하여 파일을 송신한다. (미니컴의 경우 Ctrl-A-s를 눌러 Upload 프로토콜 선택 메뉴에서 원하는 전송 프로토콜을 선택하면 파일 선택 창이 나온다. 여기서 전송할 파일을 스페이스바를 눌러 선택 한후 엔터를 눌러 전송한다.)

현재 디렉토리에 송신한 파일이 수신 되었으면 잘 컴파일된것이다. 수신한 파일이 실행파일일경우 반드시 파일모드를 실행모드로 바꿔주어야 한다. 수신시 실행모드가 무조건 바뀐다.

```
# /mnt/flash0> ls
hello_world
# /mnt/flash0> chmod +x hello_world
# /mnt/flash0> ./hello_world
Hello World!!
# /mnt/flash0>
```

6.2.4 부트시 자동 마운트

부트시 마다 일일이 mount 명령으로 유저디스크를 마운트할 필요없이 자동으로 마운트하게 할수 있다.

/etc/fstab을 코드 19와 같이 수정한다.

```
#
# fstab for NSC4000L
#
```

```
# NOSCOM Co., Ltd.
#
#
#
# /dev/mtdblock1      /          jffs  defaults    1 1
/dev/rom0             /          romfs defaults    0 0
/dev/ram0             /var        ext2  defaults    0 0
/dev/ram1             /user      ext2  defaults    0 0
/proc                 /proc      proc  defaults    0 0
/dev/mtdblock0        /mnt/flash0 jffs  defaults    0 0
/dev/mtdblock1        /mnt/flash1 jffs  defaults    0 0
[root@noscom romdisk]#
```

코드 19 /etc/fstab

6.3 16C554 확장시리얼포트

16C554는 외부 확장 시리얼 포트로서 CPU에 내장하고 있는 flow control이 되지 않는 2개의 시리얼포트에 추가하여 4개의 시리얼 FULL 포트를 사용할수있게 해 준다. 16C554와 같은 시리얼 디바이스 드라이버는 가장 대표적인 character device driver라 할수있겠다. 디바이스 드라이버를 작성하기 위해서는 작동시킬 디바이스의 스펙과 작동원리, 메모리맵등을 상세히 숙지하고 있어야 한다.

- 제조사 : Texas Instrument
- 모델명 : TL16C554A
- 특징
 - 4개의 향상된 TL15C550C ACEs Plus Steering Logic으로 구성
 - 16Byte Transmit, Receive FIFO
 - 1M bps transmit, receive operation (16Mhz)
 - 독립적인 transmit, receive control
 - 프로그램가능한 Baud-Rate 발생기

- Standard Modem Interface
- 자세한 것은 TL16C554 데이터 쉬트 참조

■ 회로

- 16 Bit I/O 모드로 구성 (인터럽트)
- 11.0592 Mhz 크리스탈 Clock 사용

CHANNEL	INTERRPUT	MEMORY MAP	CLOCK
A	24	0x04000000	11.0592 Mhz
B	23	0x06000000	
C	22	0x08000000	
D	21	0x0A000000	

표 3 16C554 메모리맵

외부인터럽트는 Rising edge를 사용하므로 I/O port 의 External Interrupt Control Register 설정을 Rising edge triggered로 맞추어 주어야 한다

6.3.1 드라이버 작성

16C554 드라이버는 ~drivers/char/serial.c를 수정하여 사용한다. 이 파일에서 조작에 필요한 레지스터가 맵핑된 메모리맵과 클럭, 인터럽트, I/O의 Signaling method를 Target 보드에 맞게 수정하고 캐쉬를 신경써주어야 한다. 맵핑된 주소에 캐쉬를 동작시키면 16C554의 레지스터에 데이터 입력할 때 그때그때 입력이 액세스가 제대로 안되기 때문에 동작을 제대로 하지 않을 수도 있기 때문이다.

우선 rs_table 의 BASE_BAUD, 메모리 맵핑주소, 인터럽트 주소값등을 수정하여 준다.

```
...
// #include <asm/serial.h> /* prevent CPU UART redefine */
...
/* 16c554 use 11.0592Mhz clock speed */
```

```

#define NSC4KL_BASE_BAUD 11059200/16

#define TL554_UART W
{0, NSC4KL_BASE_BAUD, (unsigned long)&NSC4KL554_1, INT_EINT1, STD_COM_FLAGS},W
{0, NSC4KL_BASE_BAUD, (unsigned long)&NSC4KL554_2, INT_EINT2, STD_COM_FLAGS},W
{0, NSC4KL_BASE_BAUD, (unsigned long)&NSC4KL554_3, INT_EINT3, STD_COM_FLAGS},W
{0, NSC4KL_BASE_BAUD, (unsigned long)&NSC4KL554_4, INT_EINT4567, STD_COM_FLAGS}

static struct serial_state rs_table[RS_TABLE_SIZE] = {
    TL554_UART    /* Defined in UPPER */
};
...

```

코드 20 ~/driver/serial/serial_nsc4kl.c 의 rs_table 수정

코드 18 의 수정에 따라 각각의 값을 정의하는 헤더 파일을 하나 만들어주고 이를 기존 hardware.h 에 파일에 include 시켜준다.

```

#ifndef _INCLUDE_NSC4KL_H_
#define _INCLUDE_NSC4KL_H_

#define _NSC4KL554_1 0x04000000
#define _NSC4KL554_2 0x06000000
#define _NSC4KL554_3 0x08000000
#define _NSC4KL554_4 0x0a000000

#define NSC4KL554_1    (*((volatile u16 *)_NSC4KL554_1))
#define NSC4KL554_2    (*((volatile u16 *)_NSC4KL554_2))
#define NSC4KL554_3    (*((volatile u16 *)_NSC4KL554_3))
#define NSC4KL554_4    (*((volatile u16 *)_NSC4KL554_4))

#endif

```

코드 21 ~include/asm/arch/nsc4kl.h 추가

...

```

#define __ASM_ARCH_HARDWARE_H

#include "nsc4kl.h"

#define REGBASE          0x01c00000
#define REGL(addr)      (*(volatile unsigned int *)(REGBASE+addr))
...

```

코드 22 ~include/asm/arch/hardware.h 수정

마지막으로 커널 부트시 초기화 시켜주는 rs_init() 함수의 수정이 필요하다.

```

...
/*
 * The serial driver boot-time initialization code!
 */
static int __init rs_init(void)
{
    int i, flags;
    struct serial_state * state;

    save_flags_cli(flags);
    /* External Interrupt Control Register rising edge */
    EXTINT = 0x22744442;
    /* Cache disable 0x0 ~ 0xc0000000 */
    NCACHEB1 = (((0x0C000000>>12)<<16)|(0>>12));
    restore_flags(flags);

    for( i=0, state = rs_table; i < NR_PORTS;i++, state++ ) {
        /* this machine's io_type is no virtual address */
        state->io_type = SERIAL_IO_PORT;
        state->iomem_base = (u8 *)state->port;
        state->iomem_reg_shift = 1;
    }
    ...

    /* CPU UART use minor number 0,1 */

```

```

        serial_driver.minor_start = 64 + SERIAL_DEV_OFFSET + 2;
    ...

    state->line = i+2;
    ...

```

코드 23 rs_init() 함수의 수정

6.3.2 커널옵션메뉴스크립트 추가

6.2.1과 같이 수정한 serial_nsc4kl.c를 ~/driver/serial 에 추가후 커널옵션에서 설정 할수 있도록 Config.in을 수정해야 한다. 우선 커널옵션에서 16C554를 선택할 수 있도록 ~/driver/serial/Config.in을 다음과 같이 수정해준다.

```

...
if [ "$CONFIG_SERIAL_S3C44B0" = "y" ]; then    int ' Default S3C44B0 serial
baudrate' CONFIG_S3C44B0_DEFAULT_BAUDRATE 115200
fi

    dep_bool ' NSC4000L 16C554 serial port support' CONFIG_16C554_NSC4000L
$CONFIG_ARCH_S3C44B0

    dep_tristate 'ARM AMBA serial port support' CONFIG_SERIAL_AMBA
$CONFIG_ARCH_INTEGRATOR
...

```

코드 24 Config.in 수정

그리고 커널 컴파일시 같이 컴파일 될수 있도록 ~/driver/serial/Makefile을 다음과 같이 수정한다.

```

...
obj-$(CONFIG_SERIAL_UART00) += serial_uart0.o
obj-$(CONFIG_SERIAL_S3C44B0) += s3c44b0.o
obj-$(CONFIG_16C554_NSC4000L) += serial_nsc4kl.o

include $(TOPDIR)/Rules.make
...

```

코드 25 Makefile 수정

이제 16C554 를 사용하기 위한 커널수정이 끝났다. 이제 커널옵션에서 16C554 드라이버를 선택해주고 다시 재 컴파일을 해주어야 한다.

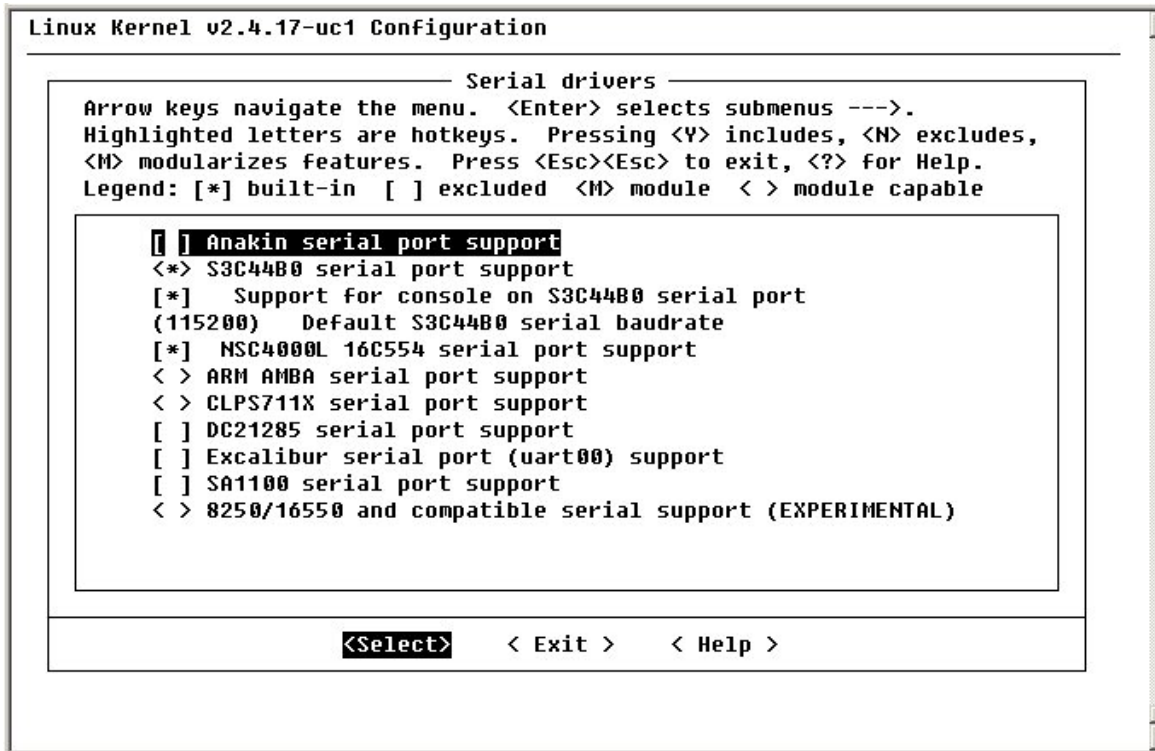


그림 23 16C554 driver 옵션

```
# make dep; make clean; make zImage
```

커널을 다시 단말기에 다운로드 해주고 다시 부팅하면서 다음과 같은 메시지가 나온다면 이상없이 잘 포팅된것이다.

```
...
Starting kswapd
JFFS version 1.0, (C) 1999, 2000 Axis Communications AB
ttyS0 at I/O 0x1d00000 (irq = 3) is a S3C44B0
ttyS1 at I/O 0x1d04000 (irq = 2) is a S3C44B0
NSC4KL Serial driver version 5.05b (2001-06-15) with no serial options enabled
ttyS02 at 0x4000000 (irq = 24) is a 16550A
```

```

ttyS03 at 0x6000000 (irq = 23) is a 16550A
ttyS04 at 0x8000000 (irq = 22) is a 16550A
ttyS05 at 0xa000000 (irq = 21) is a 16550A
block: 64 slots per queue, batch=16
RAMDISK driver initialized: 16 RAM disks of 4096K size 1024 blocksize
Blkmem copyright 1998,1999 D. Jeff Dionne
Blkmem copyright 1998 Kenneth Albanowski
...

```

그림 24 16C554 가 정상적으로 포팅된 커널 부팅메시지

6.3.3 디바이스 노드 추가

루트 파일 시스템의 /dev 에 ttyS0 ~ ttyS5 까지 6개의 노드가 존재하는지 확인한다.

보통 ttyS0 ~ ttyS3 까지만 있다면 ttyS4,5를 추가해 주어야지만 4개의 모든 16C554 포트를 사용할수 있다. 루트파일 시스템의 /dev 디렉토리에 다음과 같이 디바이스 노드를 만들어 준다.

```

# mknod ttyS4 c 4 68
# mknod ttyS5 c 4 69

```

ttyS4,5가 생성되었다면 다시 롬디스크이미지를 생성하고 단말기에 다운로드 해준다.

6.3.4 테스트

Target 보드의 16C554의 첫번째 포트(ttyS2)는 직접 TRS와 통신을 하게 되므로 하드웨어에서 전압레벨이 TTL 레벨로 고정되어 있고 나머지 포트는 MAX232 레벨로 고정되어 있다. PC는 MAX232 레벨을 사용하므로 첫번째 포트를 제외한 2,3,4 번째 포트에서 테스트를 해야 한다. 실질적으로 두번째 포트는 디버깅 포트와 함께 사용 하므로 3번째 포트를 PC와 연결하여 테스트를 해보아야 한다.

- 코드 26의 소스를 코딩하여 크로스 컴파일한다.

```
#include <termios.h>
```



```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/signal.h>
#include <sys/types.h>

#define BAUDRATE B115200
#define MODEMDEVICE "/dev/ttyS4"
#define _POSIX_SOURCE 1 /* POSIX compliant source */
#define FALSE 0
#define TRUE 1

volatile int STOP=FALSE;

void signal_handler_IO (int status); /* signal handler 함수 정의 */
int wait_flag=TRUE; /* signal을 받지 않은 동안은 TRUE */

main()
{
    int fd,c, res;
    struct termios oldtio,newtio;
    struct sigaction saio; /* signal action의 정의 */
    char buf[255];

    /* Non-blocking 모드로 시리얼 장치를 연다(read 함수 호출 후 즉각 리턴) */
    fd = open(MODEMDEVICE, O_RDWR | O_NOCTTY | O_NONBLOCK);
    if (fd <0) {perror(MODEMDEVICE); exit(-1); }

    /* install the signal handler before making the device asynchronous */
    /* 장치를 비동기 모드로 만들기 전에 signal handler */
    saio.sa_handler = signal_handler_IO;
    // saio.sa_mask = 0;
    saio.sa_flags = 0;
    saio.sa_restorer = NULL;
    sigaction(SIGIO,&saio,NULL);

```

```

/* SIGIO signal을 받을 수 있도록 한다. */
fcntl(fd, F_SETOWN, getpid());
/* file descriptor를 비동기로 만든다. (manual page를 보면
   O_APPEND 와 O_NONBLOCK만이 F_SETFL에 사용할 수 있다고 되어 있다.)
*/
fcntl(fd, F_SETFL, FASYNC);

tcgetattr(fd,&oldtio); /* save current port settings */
/* canonical 입력처리를 위한 포트 세팅 */
newtio.c_cflag = BAUDRATE | CRTSCTS | CS8 | CLOCAL | CREAD;
newtio.c_iflag = IGNPAR | ICRNL;
newtio.c_oflag = 0;
newtio.c_lflag = ICANON;
newtio.c_cc[VMIN]=1;
newtio.c_cc[VTIME]=0;
tcflush(fd, TCIFLUSH);
tcsetattr(fd,TCSANOW,&newtio);

/* loop while waiting for input. normally we would do something
   useful here */
while (STOP==FALSE) {
    printf(".Wn");usleep(100000);
    /* after receiving SIGIO, wait_flag = FALSE, input is available
       and can be read */
    if (wait_flag==FALSE) {
        res = read(fd,buf,255);
        buf[res]=0;
        printf(":%s:%dWn", buf, res);
        if (res==1) STOP=TRUE; /* stop loop if only a CR was input */
        wait_flag = TRUE;      /* wait for new input */
    }
}
/* restore old port settings */
tcsetattr(fd,TCSANOW,&oldtio);
}

```

```

/*****
 * signal handler. sets wait_flag to FALSE, to indicate above loop that
 * characters have been received.
 *****/

void signal_handler_IO (int status)
{
    printf("received SIGIO signal.\n");
    wait_flag = FALSE;
}

```

코드 26 16C554 포트 테스트용 코드 (noncan.c)

```
# arm-elf-gcc -elf2flt6 -o test554 noncan.c
```

- 에러 없이 크로스 컴파일이 성공한다면 이 실행파일(test554)을 보드의 유저 영역(/mnt/flash0)에 lrz를 이용해 다운로드한다.
- 보드의 3번째 포트(ttyS4)를 PC2의 시리얼포트와 연결하고 minicom이나 하이퍼 터미널등의 프로그램을 BAUD 115200, 8, N, 1 으로 세팅해 실행해준다.
- 보드의 test554 프로그램을 실행시키고 PC2의 터미널에서 아무 문자나 타이핑하고 엔터키를 입력했을때 보드의 터미널상에 PC에서 입력한 문자가 출력된다면 테스트 성공
- 아무 입력 없이 그냥 엔터만을 입력하면 프로그램이 종료된다.

6.4 FRAMEBUFFER

⁶ uClinux의 실행파일포맷은 Flat memory를 사용하므로 링크 옵션에서 반드시 추가해준다.

이제 보드에 붙어 있는 LCD를 사용할 차례이다. LCD는 FrameBuffer 드라이버를 사용하여 하나의 메모리처럼 조작이 가능하다. 이 FrameBuffer 드라이버 역시 커널 소스에 있는 것을 수정하여 사용하여 사용하는데 큰 어려움이 없다.

6.4.1 LCD spec

RG322002YFHLYB 320*200 LCD Module

- 320 X 200 Dot matrix LCD
- STN, ANTI GLARE
- Transflective, Wide Temperature Type
- Back Light : LED (Yellow green)

6.4.2 커널의 수정

~/drivers/video/s3c44b0fb.c 를 다음과 같이 수정

```
...

memset(cfb, 0, sizeof(*cfb) + sizeof(struct display));

fbuf = kmalloc(320*200/8, GFP_KERNEL);
if(!fbuf){
    kfree(cfb);
    goto out;
}
memset(fbuf, 0, 320*200/8);

cfb->currcon          = -1;

strcpy(cfb->fb.fix.id, "s3c44b0");
cfb->fb.screen_base    = fbuf;
cfb->fb.fix.smem_start = fbuf;
cfb->fb.fix.smem_len   = 320*200/8;
cfb->fb.fix.type       = FB_TYPE_PACKED_PIXELS;

cfb->fb.var.xres = 320;
cfb->fb.var.xres_virtual = 320;
```

```

cfb->fb.var.yres = 200;
cfb->fb.var.yres_virtual = 200;
cfb->fb.var.bits_per_pixel = 1;
cfb->fb.var.grayscale = 0;
cfb->fb.var.activate = FB_ACTIVATE_NOW;
...

cfb->fb.disp = (struct display *)(cfb + 1);

// fb_alloc_cmap(&cfb->fb.cmap, CMAP_SIZE, 0);

/*
 * Power up the LCD
 */
PDATB &= (PDATB ^ ((unsigned int)1 << 4));

LCDCON1 = ((22<<12) | (0x01<<10) | (0x01<<8) | (0x0<<7) | (0x01<<5));
LCDCON2 = (10<<21) | ((320/4-1)<<10) | (200-1);
LCDSADDR1 = (0x0<<27) | (((int)fbuf>>22)<<21) | ((int)fbuf&0x001fffff)>>1;
LCDSADDR2 = (((int)fbuf&0x001fffff)+320*200/8)>>1|(13<<21);
LCDSADDR3 = (320/16);
LCDCON1 |= 0x01;

s3c44b0fb_set_var(&cfb->fb.var, -1, &cfb->fb);
...

```

코드 27 s3c44b0fb.c 의 수정

코드 27과 같이 수정을 마쳤으면 커널 옵션 메뉴에 들어가

Character devices - - -> [*] Virtual terminal

선택해준다. 옵션 메뉴에 Console drivers 메뉴가 새로 생성되어 있을것이다.

Console drivers - - -> [*] Support Frame buffer devices

[*] S3C44B0X LCD support

선택하고 저장한 후 다시 커널 컴파일

```
# make dep; make clean; make zImage
```

부트로더를 이용하여 단말기에 커널이미지를 다시 다운로드 해준다.

6.4.3 프레임 버퍼 디바이스 노드 설정

루트파일 시스템의 /dev 디렉토리에 fb0 디바이스 노드를 추가한다.

```
# mknod fb0 c 29 0
# ln -s fb0 fb
```

(fb0 라는 이름의 Major number 29 minor number 0의 캐릭터 디바이스 드라이버 노드를 생성해 주고 fb라는 이름으로 소프트 링크를 걸어주는 것이다.)

다시 romdisk 이미지를 생성하여 단말기에 다운로드 해준다.

6.4.4 테스트

코드 28을 컴파일하여 단말기에 다운로드한다.

```
/*
 * fbpixel2.c : Frame buffer draw pixel example(using mmap)
 *
 * Copyright(C) 2002 holelee
 *
 */

#include <stdio.h>
#include <stdlib.h> /* for exit */
#include <unistd.h> /* for open/close .. */
#include <fcntl.h> /* for O_RDWR */
#include <sys/ioctl.h> /* for ioctl */
#include <sys/mman.h> /* for mmap */
#include <linux/fb.h> /* for fb_var_screeninfo, FBIOGET_VSCREENINFO */
```

```

#define FBDEVFILE "/dev/fb"

int main()
{
    int fbfd;
    int ret;
    struct fb_var_screeninfo fbvar;
    unsigned int pixel;
    int offset;
    unsigned int *pfbdata;

    fbfd = open(FBDEVFILE, O_RDWR);
    if(fbfd < 0)
    {
        perror("fbdev open");
        exit(1);
    }

    ret = ioctl(fbfd, FBIOGET_VSCREENINFO, &fbvar);
    if(ret < 0)
    {
        perror("fbdev ioctl");
        exit(1);
    }

    if(fbvar.bits_per_pixel != 1)
    {
        fprintf(stderr, "bpp is not 1\n");
        exit(1);
    }

    pfbdata = (unsigned int *) mmap(0, fbvar.xres*fbvar.yres/8,
    PROT_READ|PROT_WRITE, MAP_FIXED, fbfd, 0);

    if((unsigned)pfbdata == (unsigned)-1)

```

```

{
    perror("fbdev mmap");
    exit(1);
}

pixel = 0xF0F0F0F0;
*pfbddata = pixel;

offset = (50*fbvar.xres+64)/32;
pixel = 0xF0F0F0F0;
*(pfbddata+offset) = pixel;

offset = (100*fbvar.xres+64)/32;
*(pfbddata+offset) = pixel; /* draw pixel */

offset = (150*fbvar.xres+104)/32;
*(pfbddata+offset) = pixel; /* draw pixel */

munmap(pfbddata, fbvar.xres*fbvar.yres/8);
close(fbfd);
exit(0);
return 0;
}

```

코드 28 mmap을 이용한 LCD에 점찍기

```
# arm-elf-gcc -elf2flt -o fbpixel2 fbpixel2.c
```

생성된 실행 파일을 단말기에 lrz를 이용 다운로드 하고 실행 시킨다.

LCD 화면 세로로 50, 100, 150 dot 위치에 점선이 그려진 것이 보이면 LCD 프레임 버퍼 디바이스가 제대로 올라간 것이다.

6.5 KEYPAD

Target 보드의 유일한 입력장치는 키패드 드라이버이다. 이는 Port C를 입력으로 Port F를 출력으로하는 매트릭스 키보드 스캐닝 알고리즘을 이용하여 디바이스 드라이버를 구현한다.

NSC4000L 각각의 I/O는 부트로더인 blob에서 이미 초기화 되었다. 키패드 드라이버에서 사용하는 PORT C와 PORT F 의 초기화 루틴을 보면 코드29와 같다.

```
...

/* Port C */

//-----
// PORT C GROUP 00 = IN, 01 = OUT, 11 = Tx/Rx
// VCC VCC GPS_RXD1 GPS_RXD1 GND KO0 KO1 KO2 KO3 KO4 KO5 DISP/OFF GPS_RESET TCP_RESET
TRILED GPSLED
// 00 00 11 11 00 01 01 01 01 01 01 01 01 01 01
01 01
//-----

ldr r1, =PCONC
ldr r0, =0x0f155555
str r0, [r1]

ldr r1, =PDATC
ldr r0, =0x01f // LCD ON
str r0, [r1]

ldr r1, =PUPC
ldr r0, =0xf7e0
str r0, [r1]

...

/* Port F */

//-----
// PORT F GROUP Keyboard (00,000 = INPUT, 01,001 = OUTPUT)
// GND STLED KI2 KI1 NC NC NC KI0 VCC
// 000 001 000 000 01 01 01 00 00
//-----

ldr r1, =PCONF
ldr r0, =0x010150
```

```

str    r0, [r1]

ldr    r1, =PDATF
ldr    r0, =0x00
str    r0, [r1]

ldr    r1, =PUPF
ldr    r0, =0x100
str    r0, [r1]
...

```

코드 29 I/O PORT C,F의 초기화 루틴

PORT C의 초기화를 보면 PORT GROUP C의 PC5 ~ PC10 의 Control Register가 모두 OUTPUT으로 설정 되어 있고 pull up register는 disable시켜 놓았다. 즉 PORT C 6개의 데이터 비트가 출력용으로 사용하며 pull up되어 있지 않다는 것을 의미하며 PORT F의 경우는 PORT GROUP F의 PF1, PF5, PF6이 INPUT을 설정 되어 있고 pull up 저항 연결을 enable 시켜 놓았다. Pull up 저항이 연결되어 있으므로 PF1,5,6의 핀은 항상 high신호가 들어오게 되는 것이다.

6.5.1 키패드 드라이버 작성

qt_button.c, qt_button.h, nsc4kl_keyb_hw.h 3개의 파일을 ~/driver/input 에 복사해 넣는다. 타이눅스박스 II 의 파일들을 보드에 맞게 수정해준 것이다.

qt_button.c에는 기본적인 드라이버 초기화 파일들과 파일조작함수등 디바이스드라이버 등록에 관한 파일들이 있다.

```

...

#define BUTTON_DD_NAME  "button"

#define NUM_OF_BUTTON 18

#define POLL_EXPIRE_TIME (HZ / 10) /* 100 msec */

#define DEBOUNCE_EXPIRE_TIME (HZ / 20) /* 50 msec */

```

```

...

#ifdef CONFIG_NSC4KL_KEYPAD
static unsigned short qt_button_map[NUM_OF_QT_BUTTON_MAP] =
{
    Key_unknown,
    Key_F2, Key_Asterisk, Key_6, Key_2, Key_F6,
    Key_F3, Key_0, Key_7, Key_3, Key_F7,
    Key_NumberSign, Key_8, Key_4, Key_F8, Key_F5

};
#endif
...

#ifdef CONFIG_SA1100_GARNET
#include "garnet_keyb_hw.h"
#endif

#ifdef CONFIG_NSC4KL_KEYPAD
#include "nsc4kl_keyb_hw.h"
#endif

//-----
-
int    __init qt_button_init(void)
{
...

```

코드 30 qt_button .c 의 수정

nsc4kl_keyb_hw.h 에는 키맵이 정의 되어 있다.

```

...

static unsigned int b_map[6][3] =
    { { 1<<6, 1<<3, 1<<0},
      { 1<<7, 1<<4, 1<<1},
      { 1<<8, 1<<5, 1<<2},
      { 1<<9, 1<<10, 1<<11},

```

```

        { 1<<12, 1<<13, 1<<14},
        { 1<<15, 1<<16, 1<<17}}};

//-----
static int hw_qt_button_check( void ) // 0=valid map, else=invalid button map
{
    int i;
    unsigned int result;

    for (i = 0; i < 6; i++) {
        PDATC |= 0x07E0;
        PDATC &= (PDATC ^ ((unsigned int)1 << (10-i)));
        result = 0;

        if (!(PDATF & 0x02)) result = b_map[i][0];
        else if (!(PDATF & 0x020)) result = b_map[i][1];
        else if (!(PDATF & 0x040)) result = b_map[i][2];

        if (result)
            break;
    }
    hw_qt_button_map = result;
#ifdef 0
    if (result)
        printk("result: %x\n", result);
#endif

    return 0;
}
...

```

코드 31 nsc4kl_keyb_hw.h 의 작성

qt_button.h 에는 헤더파일이 정의 되어 있다.

...

```

#define DEF_AUTO_REPEAT_DELAY 50      // 500ms
#define DEF_AUTO_REPEAT_INTERVAL 10   // 100ms = 10Hz

/* IOCTL commands */
#define BTNIOC_GET_ATRPT_ONOFF         _IOR('B',1,int)  // 0=off, 1=on
#define BTNIOC_SET_ATRPT_ONOFF         _IOW('B',2,int)  // 0=off, 1=on
#define BTNIOC_GET_ATRPT_START_DELAY   _IOR('B',3,int)  // unit = 10ms
#define BTNIOC_SET_ATRPT_START_DELAY   _IOW('B',4,int)  // unit = 10ms
#define BTNIOC_GET_ATRPT_INTERVAL      _IOR('B',5,int)  // unit = 10ms
#define BTNIOC_SET_ATRPT_INTERVAL      _IOW('B',6,int)  // unit = 10ms
#define BTNIOC_GET_BUTTON_MAP
    _IOR('B',7,short[NUM_OF_QT_BUTTON_MAP])

...

```

코드 32 qt_button.h 의 작성

6.5.2 커널옵션메뉴스크립트 추가

menuconfig 에서 input device 의 메뉴를 추가해주기 위해 ~arch/armnommu/
config.in 수정

```

...
bool 'Support Frame buffer devices' CONFIG_FB
    source drivers/video/Config.in
endmenu
fi
#-----
#                               I n p u t
#-----

source drivers/input/Config.in

#-----
#                               M i s c   D r i v e r s
#-----

```

```
source drivers/parport/Config.in
...
```

코드 33 Input 드라이버의 추가

여기에 NSC4000L의 드라이버를 추가하기 위해 ~/driver/input/Config.in 수정

```
...
tristate 'Input core support' CONFIG_INPUT
dep_tristate ' Nsc4kl Keypad support' CONFIG_NSC4KL_KEYPAD $CONFIG_INPUT
dep_tristate ' Keyboard support' CONFIG_INPUT_KEYBDEV $CONFIG_INPUT
...
```

코드 34 ~/driver/input/Config.in 수정

컴파일을 위해 ~driver/input/Makefile 수정

```
...
obj-$(CONFIG_INPUT) += input.o
obj-$(CONFIG_NSC4KL_KEYPAD) += qt_button.o
obj-$(CONFIG_INPUT_KEYBDEV) += keybdev.o
...
```

코드 35 ~driver/input/Makefile 수정

커널 옵션에서 다음의 메뉴들을 선택해준다.

```
Input core support  - - -> <*> Input core support
                    <*> Nsc4kl Keypad support
```

옵션을 저장하고 커널을 다시 컴파일 해준다.

```
# make dep; make clean; make zImage
```

새로 컴파일한 커널이미지를 다시 보드에 다운로드한다.

6.5.3 디바이스 노드 추가

루트파일 시스템의 /dev 디렉토리에 button 디바이스 노드를 추가한다.

```
# mknod button c 81 0
```

다시 단말기에 루트 파일 시스템 다운로드하고 부팅 해준다.

6.5.4 테스트

코드 36과 같이 코딩하여 컴파일한후 단말기에 다운로드 하여 테스트한다.

```
#include <stropts.h>
#include <stdio.h>
#include <poll.h>
#include <fcntl.h>

#define BTFD "/dev/button"

void readkey(struct pollfd *pfd);

int fd;

int main()
{
    struct pollfd pfd;
    int n;

    fd = open(BTFD, O_RDONLY);

    if (fd<0) perror("Wrong FileDescriptor");

    pfd.fd = fd;
    pfd.events = POLLIN;

    for (;;) {
        n = poll(&pfd, 1, 100000);
```

```

        switch (n) {
            case -1:
                perror("Poll");
                exit(1);
            case 0:
                printf("TimeOUT\n");
                break;
            default:
                readkey(&pfd);
                break;
        }
    }
}

void readkey(struct pollfd *pfd)
{
    unsigned short key = 0;
    int n;

    if (pfd->revents & POLLIN) {
        n = read(fd, &key, sizeof(unsigned short));
        if (n>0) printf("reciece = %d, key = %d\n", n,key);
    }
}

```

코드 36 button.c

```
# arm-elf-gcc -elf2flt -o button button.c
```

모두 이상이 없이 잘 컴파일 되었다면 lrz를 이용하여 보드에 다운로드하고 실행한다.

키패드의 버튼을 누르면 누를때와 땔 때 각각 한번씩 이벤트가 일어나 해당키의 값을 출력해준다.

Chapter 7 PPPD 사용

7.1 수정 및 컴파일

MMU가 없는 uClinux에서는 fork를 사용할수 없기 때문에 vfork로 대체해주어야 한다. 그러므로 일반 ppp데몬 소스를 그냥 크로스컴파일을 하여 사용하면 안되며 uClinux용으로 패치된 ppp데몬소스를 사용해야 한다.

패치된 소스를 <http://www.wilmers.no/public/src/ppp-2.4.1-uc.tgz>에서 다운받아 압축을 풀고 컴파일을 해주기 위해서 다음과 같은 수정이 필요하다.

가) Configure 파일 수정

uClinux에서 실행되도록 컴파일 하기 위해서 약간의 수정이 필요하다.

```
...
case $system in
  Linux)
    makext="uclinux";
    ksrc="linux";
    state="known";;
...

```

코드 37 ~/configure 수정

나) Makefile.top 파일 수정

```
...
ETCDIR = $(DESTDIR)/etc/ppp

CC = arm-elf-gcc
LDFLAGS += -elf2flt
export CC LDFLAGS

# uid 0 = root

```

```
INSTALL= install
...
```

코드 38 ~/linux/Makefile.top

다) Pppstats, pppdump Makefile 수정
pppstats, pppdump의 Makefile.linux를 Makefile.uclinux로 이름을 변경하고 오브젝트파일을 링크하는 옵션에 LDFLAGS를 추가해준다.

```
...
pppstats: $(PPPSTATSRCS)
          $(CC) $(CFLAGS) $(LDFLAGS) -o pppstats pppstats.c $(LIBS)
...
```

코드 39 ~/pppstats/Makefile.uclinux

```
...
pppdump: $(OBS)
          $(CC) $(LDFLAGS) -o pppdump $(OBS)
...
```

코드 40 ~/pppdump/Makefile.uclinux

라) pppd/Makefile.uclinux 수정
pppd의 Makefile.uclinux에서는 다음과 같이 -lcrypt -lc 옵션을 더해주어야 한다.

```
...
$(EXEC): $(OBS) $(PLUGINOBJS)
          $(CC) $(LDFLAGS) -o $@ $(OBS) $(PLUGINOBJS) $(EXTRALIBS)
          $(LDLIBS$(LDLIBS-$(@))) -lcrypt -lc
...
```

코드 41 ~/ppp/Makefile.uclinux

마) main.c 의 수정

```

...
exit(1);
}
setgid(getgid());
execl("/bin/sh","sh", program, (char *)0);
error("could not exec /bin/sh: %m");
exit(99);
...

```

코드 42 ~/pppd/main.c

바) 컴파일

```
#make
```

~/pppd, ~/chat, ~/pppdump, ~/pppstats 디렉토리에 각각 pppd, chat, pppdump, pppstats 파일이 생성되어 있으면 이상없다.

7.2 실행

각각의 파일을 루트파일시스템의 /bin 에 복사해주고 chat script파일과 pppd 실행 스크립트 파일을 만들어 추가해준다.

가) Chat script 작성

```

'ABORT' 'BUSY'
'ABORT' 'ERROR'
'ABORT' 'NO CARRIER'
'ABORT' 'NO DIALTONE'
'ABORT' 'Invalid Login'
'ABORT' 'Login incorrect'
'' 'ATZ'
'OK' 'ATDT s=2'
'CONNECT' ''

```

코드 43 call

나) Pppd 실행 script 작성

```
#!/bin/sh

pppd -d -detach /dev/ttyS2 19200 crtscts defaultroute lock debug modem connect
call
```

코드 44 KTTRS_PPP

다) 보드에 call과 KTTRS_PPP 파일을 다운로드해준다.

라) 보드와 TRS를 연결하고 스크립트를 실행시킨다.

```
...

/> cd root
/root> ./KTTRS_PPP &
Couldn't restrict write permissions to /dev/ttyS2: Read-only file system
Serial connection established.
using channel 1
Using interface ppp0
Connect: ppp0 <- -> /dev/ttyS2
Warning - secret file /etc/ppp/pap-secrets has world and/or group access
sent [LCP ConfReq id=0x1 <asynctest 0x0> <magic 0x36081fc5> <pcomp> <accomp>]
rcvd [LCP ConfReq id=0x1 <pcomp> <accomp> <mru 1500> <magic 0x70f35812> <asynctest>]
sent [LCP ConfAck id=0x1 <pcomp> <accomp> <mru 1500> <magic 0x70f35812> <asynctest>]
sent [LCP ConfReq id=0x1 <asynctest 0x0> <magic 0x36081fc5> <pcomp> <accomp>]
rcvd [LCP ConfAck id=0x1 <asynctest 0x0> <magic 0x36081fc5> <pcomp> <accomp>]
sent [IPCP ConfReq id=0x1 <addr 0.0.0.0> <compress VJ 0f 01>]
rcvd [IPCP ConfNak id=0x1 <addr 172.31.46.66>]
sent [IPCP ConfReq id=0x2 <addr 172.31.46.66> <compress VJ 0f 01>]
rcvd [IPCP ConfReq id=0x1 <compress VJ 0f 01> <addr 10.23.6.1>]
sent [IPCP ConfAck id=0x1 <compress VJ 0f 01> <addr 10.23.6.1>]
rcvd [IPCP ConfAck id=0x2 <addr 172.31.46.66> <compress VJ 0f 01>]
local IP address 172.31.46.66
remote IP address 10.23.6.1
```

```
/root> ifconfig
lo          Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            UP LOOPBACK RUNNING  MTU:16436  Metric:1
            RX packets:0 errors:0 dropped:0 overruns:0 frame:0
            TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:0
            RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)

ppp0       Link encap:Point-Point Protocol
            inet addr:172.31.46.66  P-t-P:10.23.6.1  Mask:255.255.255.255
            UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1500  Metric:1
            RX packets:3 errors:1 dropped:0 overruns:0 frame:0
            TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:3
            RX bytes:42 (42.0 b)  TX bytes:48 (48.0 b)
            ...
```

그림 25 ppp가 접속되어 ip를 가져온 화면